

PROGRAMMING IN EMBEDDED C



Introduction

- The C Programming Language is a powerful, flexible and potentially portable high-level programming language.
- The C language may be used successfully to create a program for an 8-bit MCU, but to produce the most efficient machine code, the programmer must carefully construct the C Language program.
- The programmer must not only create an efficient high level design, but also pay attention to the detailed implementation.
 - Programming in C-Language greatly reduces development time.
 - C is NOT as efficient as assembly
 - A *good* assembly programmer can *usually* do better than the compiler, no matter what the optimization level – C WILL use more memory

C Language

- **Facts about the C language**

- **C is a HLL that deeply interacts with the underlying computer hardware**
- **C is an easily understood language**
- **Is a programming language widely standardized**
- **Is very productive**
- **Allows code re-use and code borrowing**
- **Allows hardware abstraction maintainability**
- **Excellent programming tool for Microcontrollers**

Assembly vs. C

- **A compiler is no more efficient than a good assembly programmer.**
- **It is much easier to write good code in C which can be converted in to efficient assembly code than it is to write efficient assembly code by hand.**
- **C is a means to an end and not an end itself.**

ANSI C for 8-Bits MCUs

Pure ANSI C is not always convenient for embedded systems because:

- Embedded systems interact with hardware. ANSI C provides extremely **crude tools** for addressing registers at fixed memory locations.
- Almost all embedded systems use **interrupts**
- ANSI C has various type **promotion rules** that are absolute performance killers to an eight-bit machine.
- Some microcontroller architectures don't have hardware support for a C stack.
- Many microcontrollers have multiple memory space

One should use standard C as much as possible... However, when it interferes with solving the problem at hand, do not hesitate to bypass it.

- Nigel Jones

Embedded vs. Desktop Programming

Main characteristics of an Embedded programming environment:

- **Limited RAM**
- **Limited ROM**
- **Limited stack space**
- **Hardware oriented programming**
- **Critical Timing (ISR, tasks, ...)**
- **Many different pointer kinds (far/near/rom/uni/paged/...)**
- **Special keywords/tokens (@, interrupt, tiny, ...)**

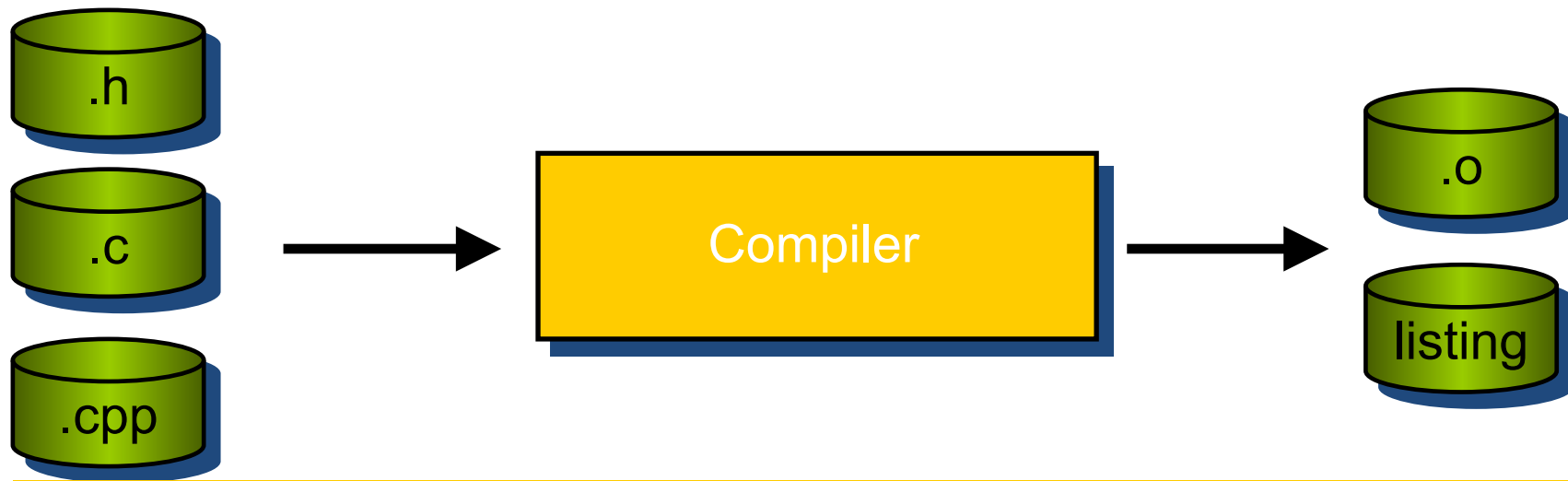
C for Microcontrollers

- Of higher level languages, C is the closest to assembly languages
 - bit manipulation instructions
 - pointers (indirect addressing)
- Most microcontrollers have available C compilers
- Writing in C simplifies code development for large projects.

Available C Compilers

- Kiel – integrated with the IDE we have been using for labs.
- Reads51 – available on web site
(<http://www.rigelcorp.com/reads51.htm>)
- Freeware: SDCC - Small Device C Compiler
(<http://sdcc.sourceforge.net/>)
- Other freeware versions ...

Compiler Requirements



- ROM-able code
- Optimized code
- Re-entrant code
- Support for Different Member in CPU Family
- Support for different memory models
- Non Obvious optimization

Compiler's little Details

While choosing a compiler, you must remember that
the **Devil** is in the **details**

Nice features that can make a huge difference:

- ✓ Inline Assembly
- ✓ Interrupt Functions
- ✓ Assembly Language Generation
- ✓ Standard Libraries
- ✓ **Startup code**

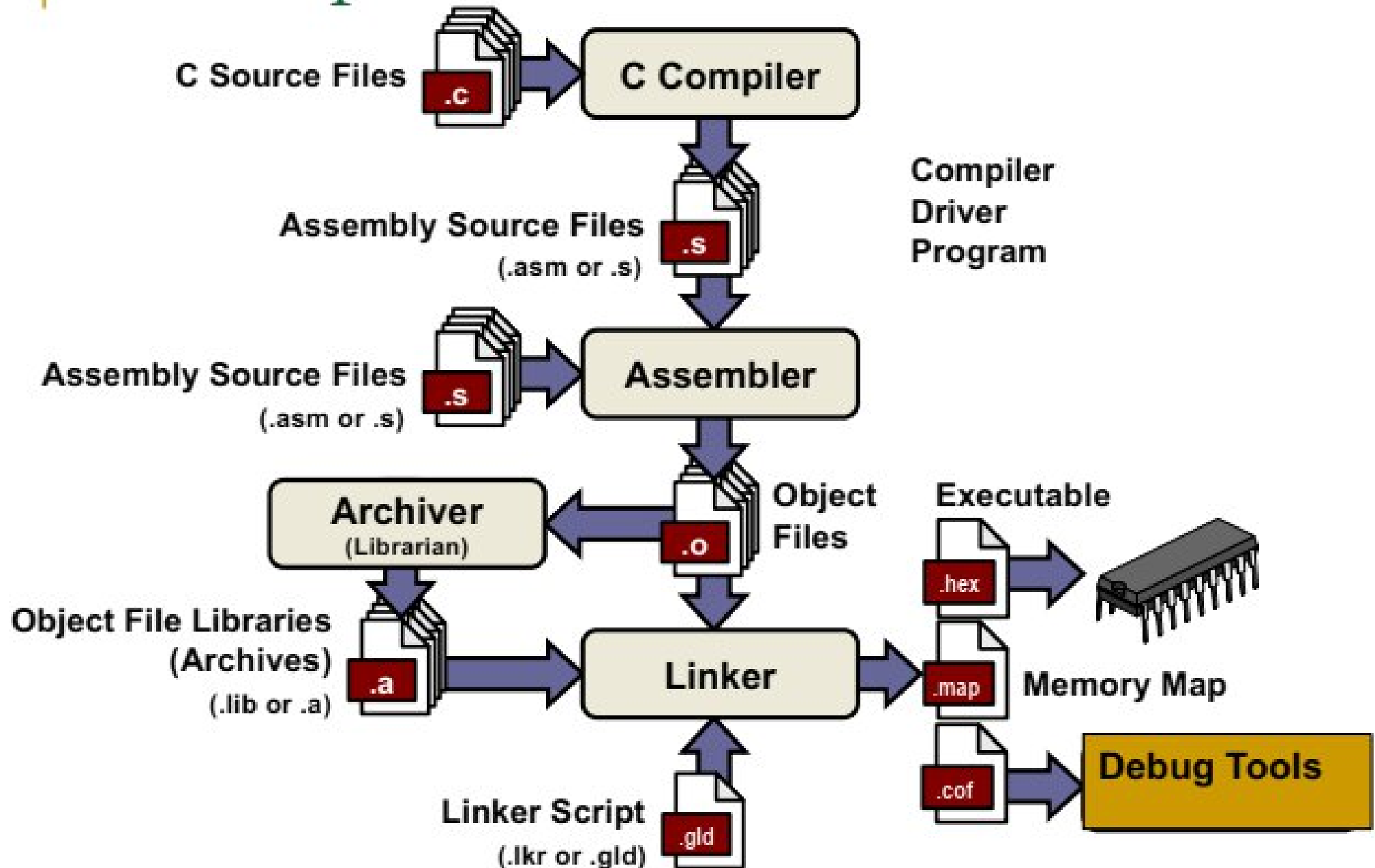
```
#include <stdio.h>

void main(void) {
    printf ("Hello World!\n");
    while(1);
}
```

- Startup code
- Standard libraries

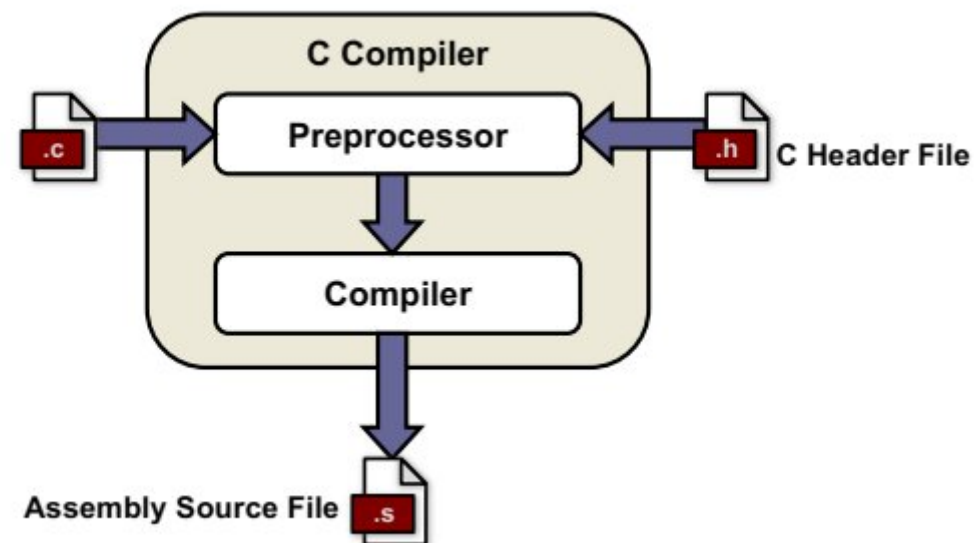
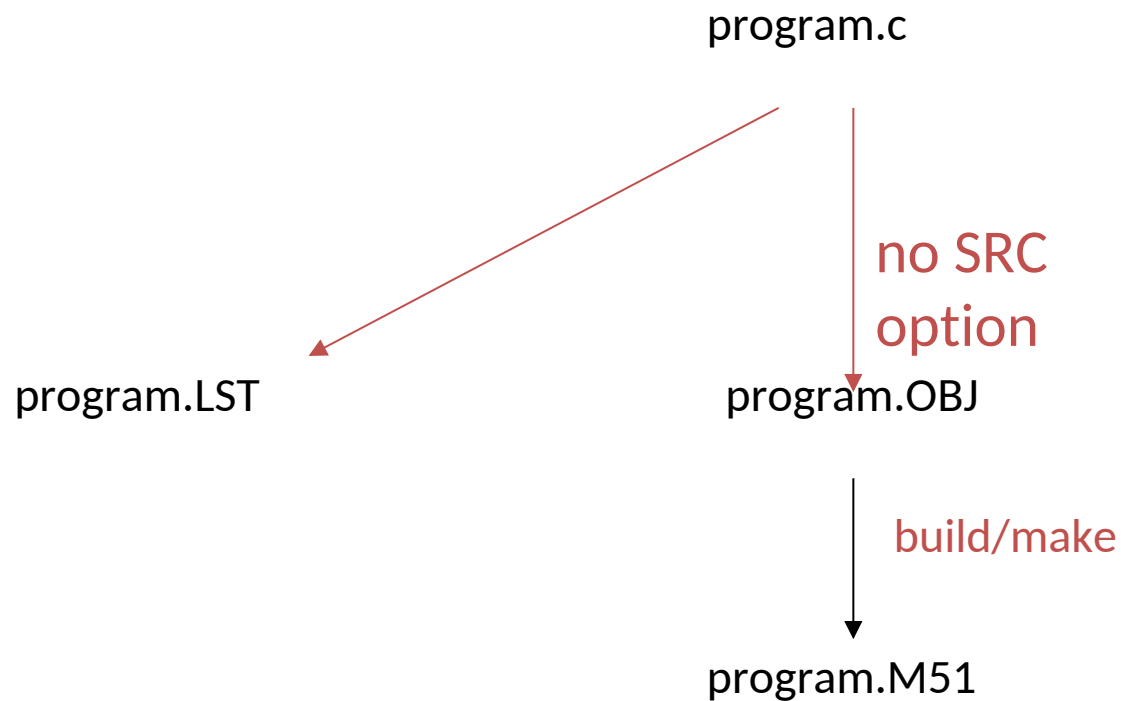
- Startup code is an extra piece of software that executes prior to main().
- A standard set of function libraries. These include various routines like printf, memcpy, strcmp, scanf and so on.

Development Tools Data Flow



Cof: Common object file format

Compilation Process (Keil)

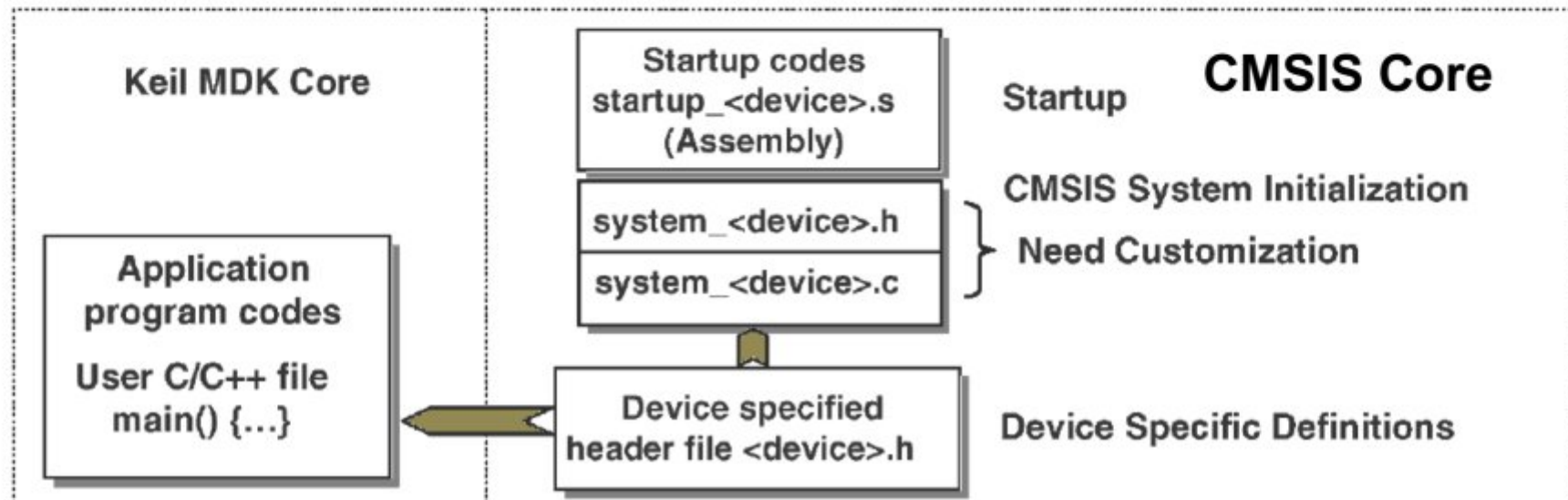


Keil C Compiler

- A **compiler** converts a high-level language to machine instructions (object code) for the target processor
- A **cross-compiler** is a compiler that runs on a processor, usually a PC, that is different from the target processor .
- An **interpreter** executes the high-level language one line at a time and does not generate an object code.
- An **assembler** converts an assembly code to machine instructions (object code) for the target processor.
- A **linker** links different software components together (e.g., main.s, start_up)
- A **debugger** is a set of hardware and software components that ensures the system operates properly,

Integrated Development Environment—Keil-MDK μ Version5

- To successfully build a basic user project with Keil - MDK, one needs to use the following two key components
 - 1. Keil- MDK Core
 - 2. CMSIS Core
- The MDK-Core provides all development tools including μ Version IDE, Compiler, and Debugger.
- The CMSIS-CORE defines the API for the Cortex-M processor core and peripherals and includes a consistent system startup code.



C Runtime Environment

- C Compiler sets up a runtime environment
 - Allocates space for stack
 - Initializes stack pointer
 - Copies values from Flash/ROM to variables in RAM that were declared with initial values
 - Clears uninitialized RAM
 - Disables all interrupts
 - Calls `main()` function (where your code starts)

Modular Programming

- Like most high level languages, C is a modular programming language (but NOT an object oriented language)
- Each task can be encapsulated as a function.
- Entire program is encapsulated in “main” function.

Basic C Program Structure

1. Compiler directives and include files
2. Declarations of global variables and constants
3. Declaration of functions
4. Main function
5. Sub-functions
6. Interrupt service routines

Example: [blinky.c](#)

A Simple Program

- All statements are terminated using “statement terminator (semicolon) `;`”
- Comments are after `//` or within `/* blab blab */`
- **Variables** can be `_MyVariable`, `MyVariable`, etc.
- There are many **keywords**: `else`, `if`, `float`, etc.

```
1 #include <stdio.h>
2 int main()
3 {
4     printf("Hello, World!\n"); // this is a comment
5     /* This is a comment block */
6     return 0;
7 }
```

Example

```
#include <stdio.h>

#define PI 3.14159

int main(void)
{
    float radius, area; // Variable Declarations

    //Calculate area of circle
    radius = 12.0;
    area = PI * radius * radius; // Variables in use
    printf("Area = %f", area);
}
```

Example

Preprocessor Directives

Header File

```
#include <stdio.h>

#define PI 3.14159 // Constant Declaration (Text Substitution Macro)

int main(void) // Void Type
{
    float radius, area; // Variable Declarations
    /* The calculations */ // Comments
    //Calculate area of circle
    radius = 12.0;
    area = PI * radius * radius; // Terminator
    printf("Area = %f", area);
}
```

Back to C Basics

- All C programs consists of:
 - Variables
 - Functions (one must be “main”)
 - Statements
- To define the SFRs as variables:
`#include <c8051F020.h>`

Variables

- All variables must be declared at top of program, before the first statement.

- Declaration includes *type* and list of variables.

Example: void main (void) {

int var, tmp;

← must go HERE!

- Types:

- **int** (16-bits in our compiler)

- **char** (8-bits)

- **short** (16-bits)

- **long** (32-bits)

- **sbit** (1-bit) ← not standard C – an 8051 extension

- others that we will discuss later

Type	Description	Bits
char	single character	8
int	integer	16
float	single precision floating point number	32
double	double precision floating point number	64

Variables

- The following variable types can be signed or unsigned:

signed char (8 bits) -128 to +127

signed short (16 bits) -32768 to +32767

signed int (16 bits) -32768 to +32767

signed long (32 bits) -2147483648 to +2147483648

unsigned char (8 bits) 0 to + 255

unsigned short (16 bits) 0 to + 65535

unsigned int (16 bits) 0 to + 65535

unsigned long (32 bits) 0 to + 4294967295

NOTE: Default is signed – it is best to specify.

Data Types in ARM

Qualifiers: **unsigned**, **signed**, **short** and **long**

Data Type	Data Size (bits)	Alignment	Data Range
bool	8	byte	0 or 1. Bits 1-7 are ignored
char	8	byte	-128 - 127(signed) or 0 - 255(unsigned)
int	32	word	-2,147,483,648 - 2,147,483,647(signed) or 0 - 4,294,967,296(unsigned)
short int	16	halfword	-32,768 - 32,767(signed) or 0 - 65,536(unsigned)
long int	32	word	same as int
long long	64	word	-9,223,372,036,854,775,808 - 9,223,372,036,854,775,807(signed) or 0 - 18,446,744,073,709,551,616(unsigned)
float	32	word	+/- 1.4023x10 ⁻⁴⁵ to 3.4028x10 ⁺³⁸ , always signed
double	64	word	+/- 4.9406x10 ⁻³²⁴ to 1.7977x10 ³⁰⁸ , always signed
long double	96	word	very large range
Storage Size			What is the storage size of a pointer? Read the book.

Data Types in Keil -ARM (1)

C/C++ Compiler (stdint.h)

Example of some standard data type definitions for intrinsic functions

Different Storage Size

Data Type	Description
int8_t	Signed Char (8-bit)
int16_t	Signed short integer (16-bit)
int32_t	Signed integer (32-bit)
uint8_t	Unsigned Char (8-bit)
uint16_t	Unsigned short integer (16-bit)
uint32_t	Unsigned integer (32-bit)
INT8_MIN	-128
INT8_MAX	127

```
// testValue
unsigned long long testValue    = 0xFFFFFFFFFFFFFFFF; // 18446744073709551615
```

```
// 1 byte -> [0-255] or [0x00-0xFF]
uint8_t      number8      = testValue; // 255
unsigned char numberChar   = testValue; // 255
```

```
// 2 bytes -> [0-65535] or [0x0000-0xFFFF]
uint16_t     number16     = testValue; // 65535
unsigned short numberShort = testValue; // 65535
```

```
// 4 bytes -> [0-4294967295] or [0x00000000-0xFFFFFFFF]
uint32_t     number32     = testValue; // 4294967295
unsigned int  numberInt    = testValue; // 4294967295
```

```
// 8 bytes -> [0-18446744073709551615] or [0x0000000000000000-0xFFFFFFFFFFFFFFFF]
uint64_t     number64     = testValue; // 18446744073709551615
unsigned long long numberLongLong = testValue; // 18446744073709551615
```

Void Data Type

Sr.No.	Types & Description
	Basic Types
1	They are arithmetic types and are further classified into: (a) integer types and (b) floating-point types.
	Enumerated types
2	They are again arithmetic types and they are used to define variables that can only assign certain discrete integer values throughout the program.
	The type void
	The type specifier void indicates that no value is available.
	Derived types
4	They include (a) Pointer types, (b) Array types, (c) Structure types, (d) Union types and (e) Function types.

Sr.No.	Types & Description
1	Function returns as void There are various functions in C which do not return any value or you can say they return void. A function with no return value has the return type as void. For example, <u>void exit (int status);</u>
2	Function arguments as void There are various functions in C which do not accept any parameter. A function with no parameter can accept a void. For example, <u>int rand(void);</u>
3	Pointers to void A pointer of type void * represents the address of an object, but not its type. For example, a memory allocation function <u>void *malloc(size_t size);</u> returns a pointer to void which can be casted to any data type.

Do not return any value

Do not accept any parameter

C Variables & Data Types

```
#include <stdio.h>
```

```
// Variable declaration:
```

```
extern int a, b;  
extern int c;  
extern float f;
```

Declare a variable at any place.

```
int main () {
```

```
/* variable definition: */
```

```
int a, b;  
int c;  
float f;
```

Defined in the main() function only

```
/* actual initialization */
```

```
a = 10;  
b = 20;
```

```
c = a + b;  
printf("value of c : %d \n", c);
```

```
f = 70.0/3.0;  
printf("value of f : %f \n", f);
```

```
return 0;
```

```
}
```

Sr.No.	Type & Description
1	char Typically a single octet(one byte). This is an integer type.
2	int The most natural size of integer for the machine.
3	float A single-precision floating point value.
4	double A double-precision floating point value.
5	void Represents the absence of type.

Global & Local Variables

- A **global variable** is a **variable** that is declared outside all functions.
 - ▣ A **global variable** can be used in all functions.
- A local **variable** can only be used in the function where it is declared.
- Example of global variable: As you can see two **global variables** are declared, A and B. These **variables** can be used in main() and Add().

```
#include<stdio.h>

// Global variables
int A;
int B;

int Add()
{
    return A + B;
}

int main()
{
    int answer; // Local variable
    A = 5;
    B = 7;
    answer = Add();
    printf("%d\n", answer);
    return 0;
}
```

Defining I/O Registers

- **One common and very useful form is:**

```
#define PortA ( * ( volatile unsigned char * ) 0x0000 )
```

This makes `PortA` a variable of type `char` at address `0x0000`

- **An easier way to do this is:**

```
volatile unsigned char PortA @0x0000;
```

This is a compiler-specific syntax... It is *more readable*, but we pay the price loosing *portability*.

Data Type Selection

There are **3 Rules** for Data Type Selection on 8-bit MCUs:

- Use the **smallest** possible type to get the job done.
- Use **unsigned** type if possible.
- Use **casts** within expressions to reduce data types to the minimum required.

Variables

Variables are names for storage locations in memory

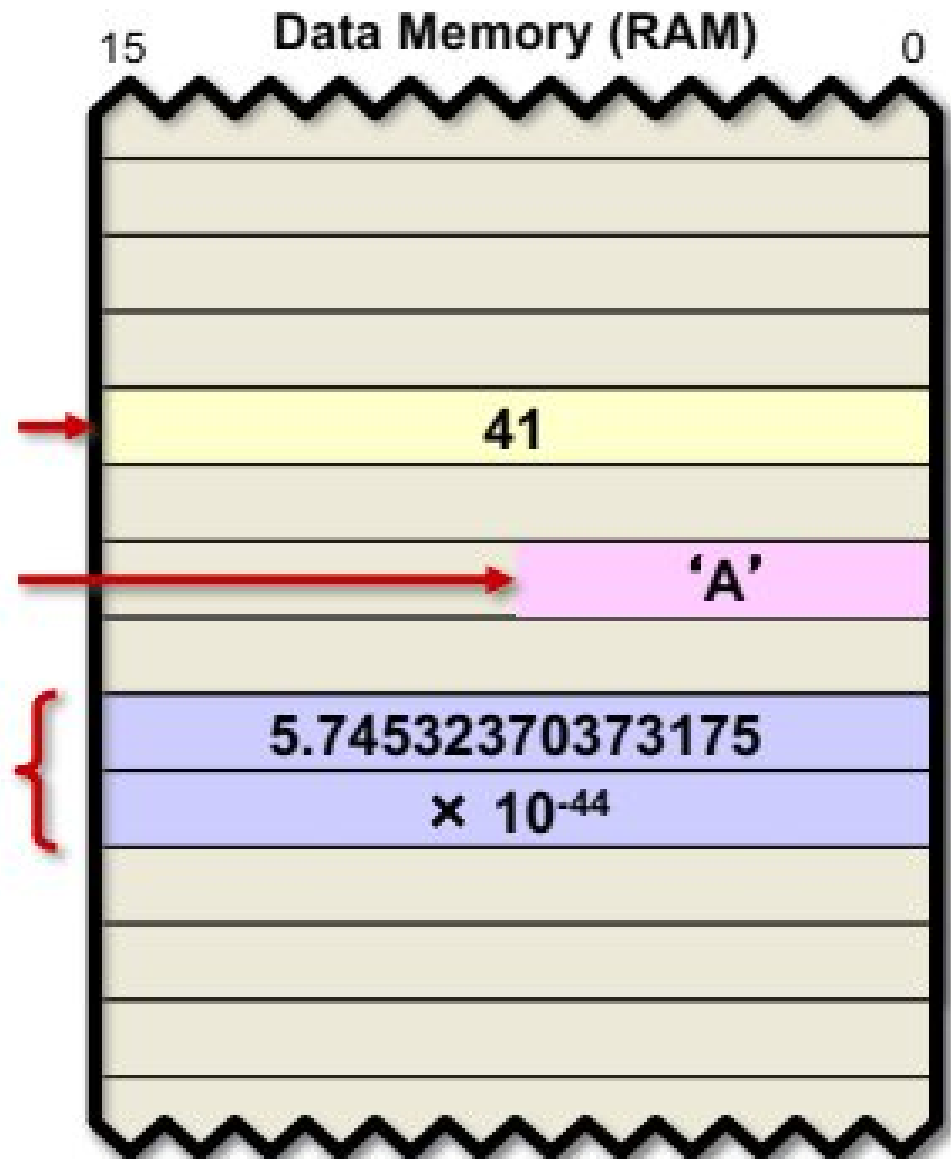
Variable declarations consist of a unique identifier (name)...

```
int warp_factor;
```

```
char first_letter;
```

```
float length;
```

Variable Storage Size



Data Types

Every variable/function must have a specific data

Data types in c refer to an extensive system used for declaring variables or functions of different types. The type of a variable determines how much space it occupies in storage and how the bit pattern stored is interpreted. The types in C can be classified as follows –c

Sr.No.	Types & Description
1	Basic Types They are arithmetic types and are further classified into: (a) integer types and (b) floating-point types.
2	Enumerated types They are again arithmetic types and they are used to define variables that can only assign certain discrete integer values throughout the program.
3	The type void The type specifier <i>void</i> indicates that no value is available.
4	Derived types They include (a) Pointer types, (b) Array types, (c) Structure types, (d) Union types and (e) Function types.

Modifiers

Three key words, together, allow us to write not only better code, but also tighter code:

- ✓ **static**
- ✓ **volatile**
- ✓ **const**

Static Variables

Static variables are initialized during the compiling and not runtime. Therefore, they are only initialized locally or globally just once

- **When applied to variables, static has two primary functions:**
 - **The variable doesn't disappear between successive invocations of a function.**
 - **When declared at the module level, it is accessible by all functions in all the module, but by no one else.**

Note:

These variables won't be stored in the Stack.

Static variable Example

```
FILE1.c

#include <FILE2.h>    includes functions
                     contained in file
FILE2.c

void main (void){

    MyFunction();     included in FILE2.c

    MyFunction();     included in FILE2.c

}
```

Before entering to
MyFunction the **first**
time, **myVar = 0**

Before entering to
MyFunction for the
second time:
myVar = 1

```
FILE2.c

void MyFunction (void){  Definition of
                          MyFunction
                          in FILE2.C

    static char myVar = 0; local variable
                          declared static

    myVar = myVar + 1;

}
```

The **static** Variable
keeps its value even
though **myVar** is **local**

Example of Static Variable in C

What are the values of y in each case?

Is X local or global variable?

C Code X is defined globally

```
int foo();

int main(void) {
    int y;
    y = foo(); // y = ?
    y = foo(); // y = ?
    y = foo(); // y = ?
    while(1);
}

int foo() {
    int x = 5;
    x = x + 1;
    return(x)
}
```

X is initialized during the runtime. Thus, every time the function is called it is set to five.

C Code X is a static variable

```
int foo();

int main(void) {
    int y;
    y = foo(); // y = ?
    y = foo(); // y = ?
    y = foo(); // y = ?
    while(1);
}

int foo() {
    static int x = 5;
    x = x + 1;
    return(x)
}
```

In this case the initialization happens during the compile time not the runtime. Thus, X is initialized only one!

Static Functions

- **Features:**

- Only callable by other functions within its module.
- Good structured programming practice.
- Can result in smaller and/or faster code.

- **Advantages:**

Since the compiler knows at compile time exactly what functions can call a given static function, it may **strategically place** the static function such that may be called using a short version of the call or jump instruction.

Volatile Variables

§ Advantages:

- Since the compiler knows at compile time exactly what functions can call a given static function, it may **strategically place** the static function such that may be called using a short version of the call or jump instruction.
- In embedded systems, there are two ways this can happen:
 - Via an interrupt service routine
 - As a consequence of hardware action



It is **very good** practice to declare **volatile** all Peripheral Registers in embedded devices.

Volatile Variable

- Sometimes values in registers are reused – this often happens when the compiler attempts to optimize
- Often times we need to make sure such stale data is avoided – thus we use volatile variables
- When accessing data of a memory-mapped I/O register we should always use volatile
- In general, we always use volatile type to access hardware registers

```
volatile uint32_t *in_reg_c = (volatile uint32_t *) GPIOC_IDR;
```

Note: volatile variables are global so they can be accessed by interrupt handlers or by multiple threads!

Where to use volatile variables

<https://embedds.com/using-volatile-keyword-in-embedded-code/>

- volatile is always used on global variables that are accessed from interrupt service routines. Otherwise code won't work
- Where to use volatile variables
 - Ideally you may think, why not use volatile everywhere if it is safe. Matter of fact is that your code would work fine with all volatiles, but it would be inefficient with lots of overhead. Hopefully you noticed the situation when you write a code and compile it without optimization "-O0" it works fine without any volatiles. This means that code is compiled with all overheads it takes much more space and may be executed much slower. But once you include some level of optimization code stops working and as a beginner you may start wondering what's a problem – code looks fine as there is no logical errors and so on. Your interrupts don't do what they are supposed to do, you get in to infinite loops, etc. This happens because compiler optimizer does some initial code analyze and finds that variable is once initialized and doesn't seem to be changed. Optimizer isn't that smart to predict interrupt behavior or other event like RTOS task. So compiler optimizes the variable out leaving it with predefined value and may not check it's contents even if you would expect it to.
 - In that case, declaring such variable as volatile rings the bell to optimizer not to optimize it.

<https://tapit.vn/y-nghia-cua-tu-khoa-volatile-trong-lap-trinh-nhung-c/>

Volatile là gì?

- Volatile có nghĩa là không dự đoán được. Một biến sử dụng với volatile qualifier có nghĩa là nói với compiler là biến này có thể sẽ được thay đổi ở bất kì chỗ nào

Cú pháp khai báo

Biến: *int volatile var = 0;*

Con trỏ: *int volatile *ptr;*

Sử dụng Volatile

Chúng ta sẽ sử dụng volatile khi khai báo biến trong các trường hợp dưới đây:

- Con trỏ trỏ tới địa chỉ của các thanh ghi trong bộ nhớ (Memory - mapped peripheral register)
- Biến toàn cục dùng chung cả trong và ngoài ngắt (ISR Routine)
- Biến toàn cục dùng chung trong nhiều task trong ứng dụng multi-thread

● Tổng kết

Những lỗi gây ra do không sử dụng volatile có thể phát hiện ra sớm, nhưng với những trường hợp như sử dụng biến trong ngắt thì có thể rất lâu sau mới phát sinh lỗi. Vì vậy hãy sử dụng volatile bất cứ khi nào có thể do risks :)

Sử dụng Volatile

- **Memory - mapped peripheral register**

Xem xét đoạn code sau

//gan con trỏ ptr tới thanh ghi trạng thái có địa chỉ 0x1234

*uint32_t *ptr = (uint32_t *) (0x1234);*

//vòng lặp kiểm tra nếu giá trị thanh ghi khác 0 thì thoát

*while (*ptr == 0);*

Dòng code đầu tiên khai báo con trỏ ptr. Giả sử giá trị thanh ghi tại thời điểm này là 0 tức *ptr = 0. Khi build tối ưu, compiler có thể sẽ không nhìn thấy giá trị *ptr được sửa ở đâu nữa nên nó sẽ coi như đây là vòng lặp vô hạn luôn (do giá trị *ptr lúc đầu = 0). Do đó để tránh trường hợp trên chúng ta cần khai báo volatile cho biến con trỏ ptr

*uint32_t volatile *ptr;*

Nếu bạn mở file .h định nghĩa các thanh ghi của một vi điều khiển sẽ thấy họ sử dụng từ khóa volatile

Sử dụng Volatile

- **ISR Routine**

Xem xét đoạn code

```
uint8_t flag = 0;
void main(void) {
    while (!flag) {
        //do something
    }
}

void ISR_UART(void) {
    //nhan ki tu tu UART
    char c = UART_GetChar();
    if (c == 'A') {
        flag = 1;
    }
}
```

Chúng ta thường hay sử dụng đoạn code như trên khi có ngắt sẽ set một cờ (flag) và sau đó trong hàm main() sẽ kiểm tra cờ để thực hiện một hành động nào đó. Tuy nhiên compiler không hề nhìn thấy là giá trị của flag có thể bị thay đổi ở trong ngắt, do đó nó tối ưu dẫn đến while(!flag) có thể trở thành một vòng lặp vô hạn. Do vậy trong trường hợp này chúng ta cũng nên thêm volatile vào trước biến flag

```
uint8_t volatile flag = 0;
```

Sử dụng Volatile

- **Multi-thread**

```
uint8_t cnt;  
void taskA() {  
    cnt = 0;  
    while (!cnt) {  
        //do something  
    }  
}  
void taskB() {  
    cnt++;  
}
```

Tương tự như 2 phần trên, compiler không biết là giá trị của biến cnt có thể thay đổi ở bên task B nên khi tối ưu sẽ dẫn đến việc chạy không đúng như ý muốn. Một nhầm lẫn là tưởng sử dụng mutex để bảo vệ dữ liệu dùng chung sẽ giải quyết được vấn đề này.

```
int cnt ;  
task_a() {  
    take(mutex);  
    cnt += 1;  
    give(mutex);  
}  
task_b() {  
    cnt = 0;  
    take(mutex);  
    if (cnt == 0) ... else ...;  
    give(mutex);  
}
```

Trong đoạn code trên, compiler không biết cnt có thể thay đổi nên nó có thể lờ đi cả đoạn code if....else.

Volatile variables are **never** Optimized

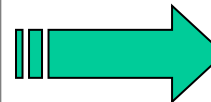
- Access to variables defined as volatile are never optimized by the compiler !!!



```
volatile unsigned char PORTA @0x00;
volatile unsigned char SCS1 @0x16;
unsigned char value;

void main(void) {

    PORTA = 0x05;          /* PORTA = 00000101 */
    PORTA = 0x05;          /* PORTA = 00000101 */
    SCS1;
    value = 10;
}
```



Without Volatile keyword

```
MOV    #5,PORTA
LDA     #10
STA     @value
```

With Volatile keyword

```
MOV    #5,PORTA
MOV    #5,PORTA
LDA     SCS1
LDX     #10
STX     @value
```

Volatile variables are **never** Optimized

- `volatile` specifies a variable whose value may be changed by processes outside the current program
- One example of a volatile object might be a buffer used to exchange data with an external device:

```
int
check_iobuf(void)
{
    volatile int iobuf;
    int val;

    while (iobuf == 0) {
    }
    val = iobuf;
    iobuf = 0;
    return(val);
}
```

if `iobuf` had not been declared `volatile`, the compiler would notice that nothing happens inside the loop and thus eliminate the loop

Volatile variable Example

```
/* MC68HC908GP20/32 Official Peripheral Register Names */

volatile unsigned char PORTA    @0x0000; /* Ports and data direction */
volatile unsigned char PORTB    @0x0001;
volatile unsigned char PORTC    @0x0002;
volatile unsigned char PORTD    @0x0003;
volatile unsigned char PORTE    @0x0008;

volatile unsigned char DDRA     @0x0004; /* Data Direction Registers */
volatile unsigned char DDRB     @0x0005;
volatile unsigned char DDRC     @0x0006;
volatile unsigned char DDRD     @0x0007;
volatile unsigned char DDRE     @0x000C;

volatile unsigned char PTAPUE   @0x000D; /* Port pull-up enables */
volatile unsigned char PTCPU   @0x000E;
volatile unsigned char PTDPUE   @0x000F;
```

Decoding the C Program

``clk_reg`` points to an object qualified with ``uint32_t`` - `uint32_t` = unsigned integer 32-bit type

```
volatile uint32_t *clk_reg = (volatile uint32_t *)RCC_AHB1ENR;  
*clk_reg |= 0x5; // Turn on GPIOA and GPIOC
```

Let's take it one step at the time:

```
0x40000000
```

is your memory address.

```
(uint32_t *)0x40000000
```

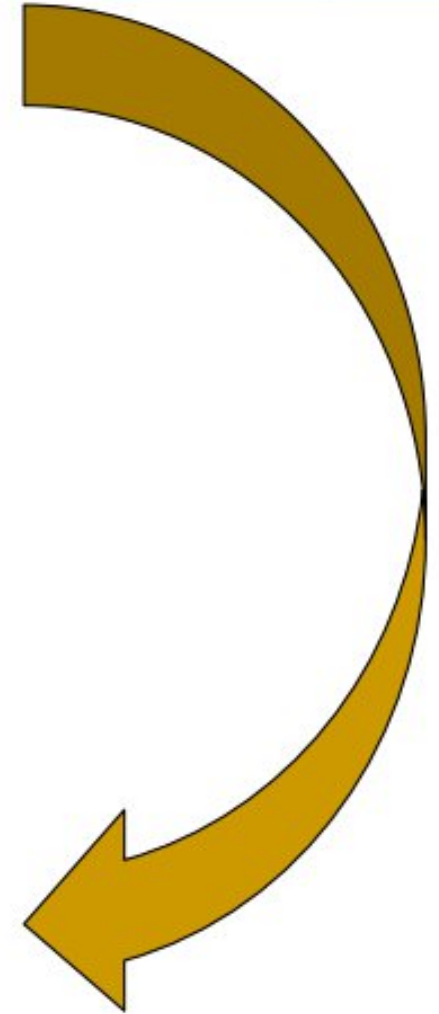
casts this to a pointer to that memory address, of type `uint32_t`, meaning 32 bit without sign.

```
(volatile uint32_t *)0x40000000
```

`volatile` means, basically, "hey compiler, don't do any optimization; I really want to go every time to that memory address and fetch it, without any prefetch or anything particular".

```
*((volatile uint32_t *)0x40000000)
```

means: take the value contained at the address identified by that pointer, so the four bytes starting at `0x40000000`.



Const variables

The Declaration **const** is applied to any variable, and it tells the compiler to store it in ROM code.

The compiler keeps the program memory address of that location. Since it is in ROM, its value cannot be changed. An initialization value must be declared.

Example:

```
const double PI = 3.14159265;
```

- `const` is used with a datatype declaration or definition to specify an unchanging value
 - Examples:

```
const int five = 5;  
const double pi = 3.141593;
```

- `const` objects may not be changed
 - The following are illegal:

```
const int five = 5;  
const double pi = 3.141593;  
  
pi = 3.2;  
five = 6;
```

Const constrains

- Some compilers create a genuine variable in **RAM** to hold the const variable. On RAM-limited systems, this can be a significant penalty.
- Some compilers, like CodeWarrior, store the variable in **ROM**. However, the “read only” variable is still treated as a variable and accessed as such, typically using some form of indexed addressing (16-bit). Compared to immediate addressing (8-bit), this method is normally much slower.

Const volatile variables

Can a variable be both, const and volatile? **Yes!**

This modifier form should be used on any memory location that can change unexpectedly (volatile) **and** that is read-only (const).

The most obvious example of this is a hardware status register:

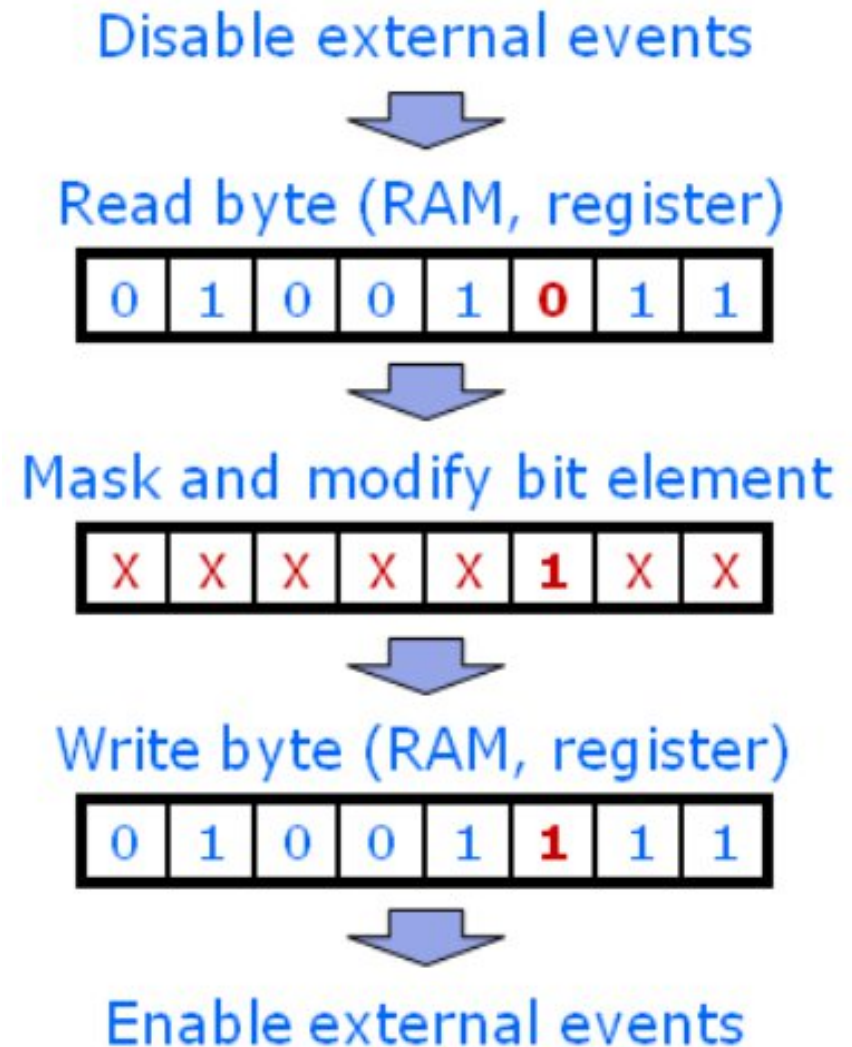
```
        /* SCI Status Register */  
const volatile unsigned char SCS1    @0x0016
```

const and volatile can be used together

- Yes. The const modifier means that this code cannot change the value of the variable, but that does not mean that the value cannot be changed by means outside this code. For instance, in the example in FAQ 8, the timer structure was accessed through a volatile const pointer. The function itself did not change the value of the timer, so it was declared const. However, the value was changed by hardware on the computer, so it was declared volatile. If a variable is both const and volatile, the two modifiers can appear in either order. `
- An input-only buffer for an external device could be declared as const volatile (or volatile const, order is not important) to make sure the compiler knows that the variable should not be changed (because it is input-only) and that its value may be altered by processes other than the current program

Bit: READ-MODIFY-WRITE

- 8051 and the earlier ARM7, ARM9 CPUs were only able to perform bit manipulations on SRAM and peripheral memory locations by using **AND** and **OR** operations. This requires a **READ-MODIFY-WRITE** operation which is expensive in terms of cycles taken to set and clear individual bits and the overall code space required for each bit manipulation.
- Using Read-Modify-Write operations to set bits works fine when you're doing one thing at a time, but problems can arise when an application is doing multiple things concurrently. For example, what happens if an interrupt occurs between the read and modify operations that changes the value in the register? The new value will get overwritten. This **race-condition** could lead to undesired behavior.

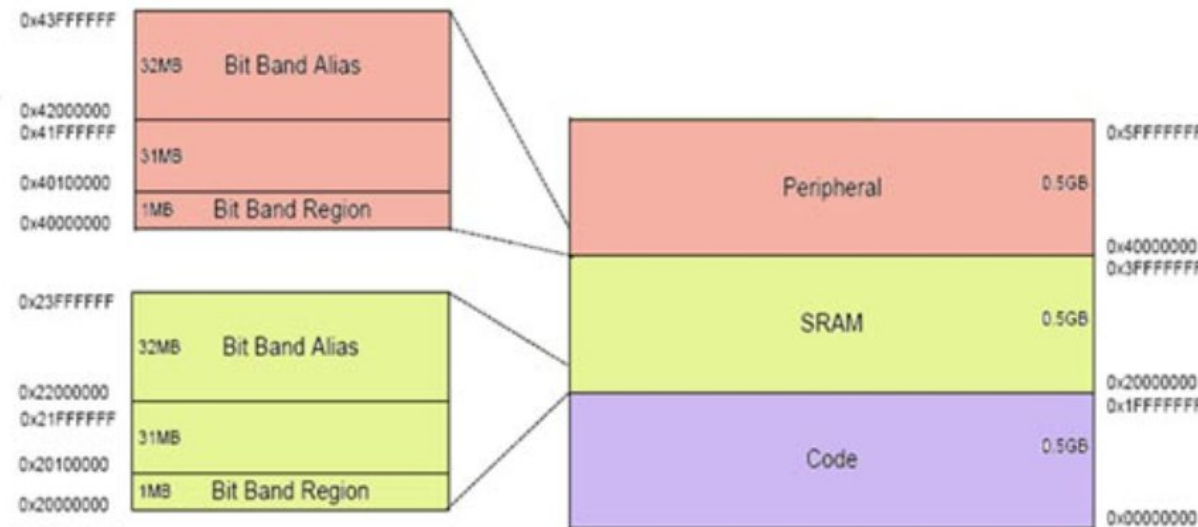
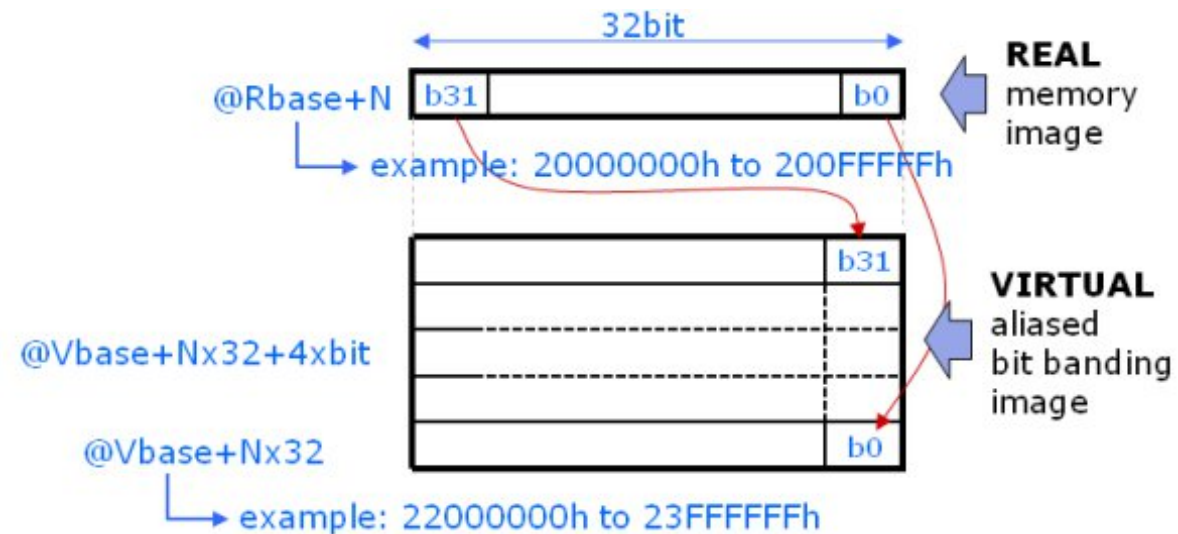


Bit Banding, Atomic Bit Manipulation

- Atomic bit manipulation -- read/modify/write in a single instruction
- Bitwise operation
- Bit banding (Bit addressable)
- Why?
 - *Atomic bit operations are very fast, since they perform the operation using a single machine instruction without disabling interrupt*
 - *Atomic bit operations are also necessary for efficient real-time operating-system (RTOS) support. This new design significantly improves on bit-handling performance.*
 - *Bit manipulation for flags and peripheral control is needed for efficient microcontroller operation. The Cortex M3 adds atomic bit operations through the use of "bit banding," where a block of memory maps to bits instead of words.*
 - *It is possible atomic bit manipulation with set/reset and reset registers. So to set one bit in port you don't have to read-modify-write port value. This is a dangerous situation where interrupt may occur in the middle of the event. And surely registers can also be bit manipulated with bit banding functionality.*

Bit banding for Atomic Bit Manipulation

- To overcome this limitation, possible to introduce a dedicated bit set and clear instructions, or a full Boolean processor, but increase the size and complexity of the Cortex CPU. Instead, a technique called bit banding allows direct bit manipulation on sections of the peripheral and SRAM memory spaces, without need for any special instructions.
- The bit addressable regions are composed of bit band region (up to 1Mbyte of real memory or peripheral registers) and bit band Alias region up to 32Mbyte of the memory map. Bit banding works by mapping each bit in 1MB bit band region to a 32-bit word address (1MB mapped to 32MB) in the Alias region. So by setting and clearing the aliased word address we can set and clear bits in the real memory.



Bit Banding is supported over the first 1Mb of the SRAM and Peripheral regions. This covers all the resources of the STM32.

Formula to calculate the alias address

Address in the bit band alias region = Bit band alias base address + bit word offset
Where bit word offset = Byte offset from bit band base X 0x20 + bit number x 4

This is much easier than it may look at first glance. For a practical example, the GPIO output data register is written to in order to set and clear individual IO lines. The physical address of the Port B output register is 0x40010C0C. In this example we want to be able to set and clear bit eight of this word using the above formula.

Word address	= 0x40010C0C
Peripheral bit band base	= 0x40000000
Peripheral bit band Alias base	= 0x42000000
Byte offset from bit band base	= 0x40010c0c – 0x40000000 = 10c0c
Bit word offset	= (0x10c0c x 0x20) +(8x4) = 0x2181A0
Bit Alias address	= 0x42000000 + 0x2181A0 = 0x422181A0

We can now create a pointer to this address using the following line of C:

```
#define PortBbit8    (*((volatile unsigned long *) 0x422181A0 ))
```

This pointer can then be used to set and clear the IO port bit:

```
PB8 = 1;           //led on
```

Which generates the following assembly instructions:

```
MOVS    r0, #0x01
LDR      r1, [pc, #104]
STR      r0, [r1, #0x00]
```

Switching the LED off:

```
PB8 = 0; //led off
```

Generates the following assembly instructions:

```
MOVS    r0, #0x00
LDR     r1, [pc, #88]
STR     r0, [r1, #0x00]
```

Both the set and clear operations take three 16-bit instructions and on the STM32 running at 72 MHz these instructions are executed in 80nsec. Any word in the peripheral and SRAM bit band regions can also be directly addressed word-wide so we could perform the same set and clear using the more traditional AND and OR approach:

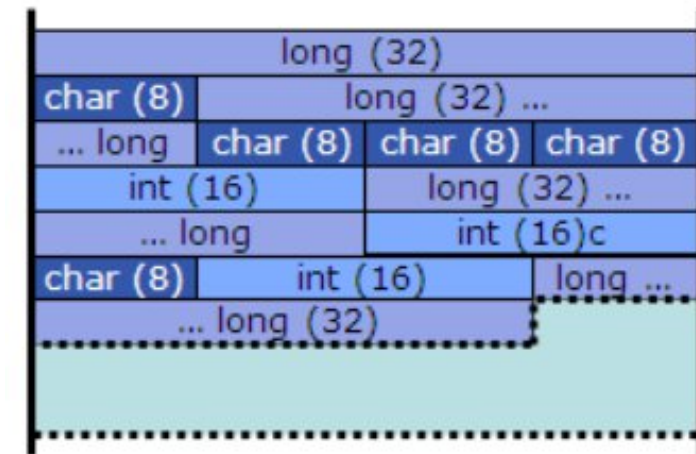
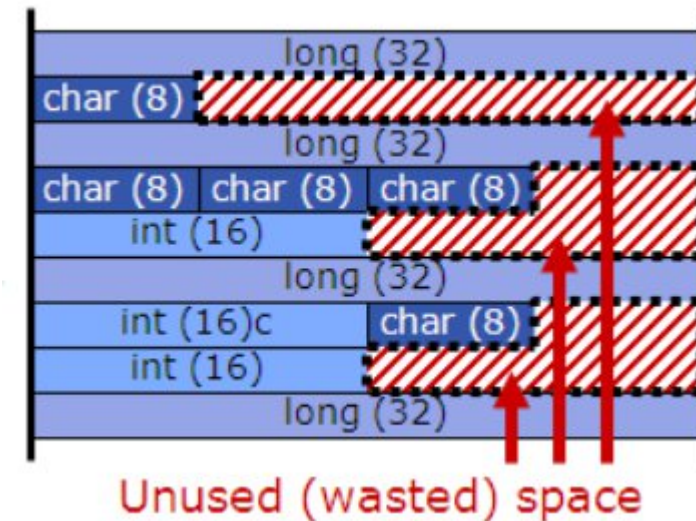
```
GPIOB->ODR |= 0x00000100;           //LED on
LDR     r0, [pc, #68]
ADDS    r0, r0, #0x08
LDR     r0, [r0, #0x00]
ORR     r0, r0, #0x100
LDR     r1, [pc, #64]
STR     r0, [r1, #0xC0C]
```

```
GPIOB->ODR &=!0x00000100;           //LED off
LDR     r0, [pc, #40]
ADDS    r0, r0, #0x08
LDR     r0, [r0, #0x00]
MOVS    r0, #0x00
LDR     r1, [pc, #40]
STR     r0, [r1, #0xC0C]
```

Now each set and clear operation takes a mixture of 16 and 32-bit operations, which take a minimum of 14 bytes for each operation and at the same clock frequency take a minimum of 180 nSec. If you consider the impact of bit banding on a typical embedded application that sets and clears lots of bits in the peripheral registers and uses semaphores and flags in the SRAM, you are very clearly going to make significant savings in both code size and execution time and it is all handled in the STM32 header file for you.

Unaligned Memory Accesses

- The ARM7 and ARM9 instruction sets are capable of accessing byte, half word and word signed and unsigned variables. This allows the CPU to naturally support integer variables without the need for the sort of software library support typically required in 8 and 16-bit microcontrollers. However, the earlier ARM CPUs do suffer from a disadvantage in that they can only do word or half-word aligned accesses. This restricts the compiler linker in its ability to pack data into the SRAM and some valuable SRAM will be wasted. (This can be as much as 25% depending on the mix of variables used.)
- The Cortex CPU has addressing modes for word, half-word and byte, but is able to make unaligned memory accesses. This gives the compiler linker complete freedom to order the program data in memory. The additional bit banding support on the Cortex CPU allows program flags to be packed into a word or half-word variable rather than using a byte for each flag.



The Cortex-M3 can make unaligned memory accesses, which ensures that the SRAM is efficiently used.

Macro

- Macro is a group of instructions. The Macros in Microprocessor assembler generates the code in the program each time where the macro is 'called'. Macros can be defined by MACRO and ENDM assembler directives. Creating macro is very similar to creating a new opcode that can be used in the program.

```

DIV16 MACRO Result, X, Y
; Store into Result the signed result of X / Y
; Calculate Result = X / Y
; (all 16-bit signed integers)
; Destroys Registers AX,DX

MOV  AX, X      ; Load AX with Dividend
CWD             ; Extend Sign into DX
IDIV Y          ; Signed Division
MOV  Result, AX ; Store Quotient
ENDM
    
```

- Example: Using the macro in a program

```

; Variable Section
varX1    DW    20
varX2    DW    4
varR     DW    ?

; Code Section
DIV16 varR,varX1,varX2
    
```

- General Format

```

MACRO_NAME MACRO Param1,Param2,...,ParamN
LOCAL MyLabel
Your Code ...
... Param1 ...
... Param2 ...
Your Code ...
JMP MyLabel
Your Code ...

MyLabel:
... ParamN ...
Your Code ...
ENDM
    
```

Procedure	Macro
1. Accessed by CALL and RET instruction during program execution.	1. Accessed during assembly with name given to macro when defined.
2. Machine code for instructions is put only once in the memory.	2. Machine code is generated for instructions each time when macro is called.
3. With procedures less memory is required.	3. With macros more memory is required.
4. Parameters can be passed in registers, memory locations, or stack.	4. Parameters passed as part of statement which calls macro.

The **#define** directive defines a preprocessor macro.

```
#define macro-name <[>(arg<[, arg ...<]>)<]> replacement-text
```

You may use **#define** to create function-like macros with or without arguments. Macros are syntactically similar to function calls. When a defined macro is encountered in the source file, the *macro-name* and any arguments are replaced by the *replacement-text*. For example:

```
#define my_macro(a,b,c) a+b+c

int func (int x, int y, int z)
{
    return(my_macro(x,y,z));
}
```

appears as follows after macro expansion:

```
int func (int x, int y, int z)
{
    return(x+y+z);
}
```

Refer to [Macros](#) for a complete description of how to define and use macros in your C programs.

Macro in 8051/AVR

For example, given the following **macro** definitions

```
#define ALPHA 0xff /* mask off lower bits */  
#define SUM(a,b) a+b
```

the following expression

```
int x = 0x3def & ALPHA;
```

would be replaced with

```
int x = 0x3def & 0xff /* mask off lower bits */ ;
```

and this expression

```
int i=SUM(2,3);
```

would be replaced with

```
int i=2+3;
```

This use of **#define** can also enhance the readability of a program by allowing a function to be “re-declared.” Let’s assume a function called *set_row_col()* places the cursor at a position on a four-row by twenty-character LCD display based on two parameters, a *row* and a *column*:

```
void set_row_col(char row, char column)  
{  
    . . .  
}
```

With the use of a **macro** to form a re-declaration, the positions on the LCD display can be referred to as an *x* and *y* coordinate instead of a row and column:

```
#define goto_xy(x,y) set_row_col(y,x)
```

Another possibility is a re-declaration that can use a character position to set the cursor:

```
#define set_char_pos(p) set_row_col((p/20),(p%20))
```


DATA TYPES

A good understanding of C data types for 8051 can help programmers to create smaller hex files

- Unsigned char

- Signed char •

Unsigned int •

Signed int

- Sbit (single bit) •

Bit and sfr



DATA TYPES

Unsigned char

The character data type is the most natural choice

- 8051 is an 8-bit microcontroller

Unsigned char is an 8-bit data type in the range of 0 - 255 (00 - FFH)

- One of the most widely used data types for the 8051

Counter value

ASCII characters

C compilers use the signed char as the default if we do not put the keyword unsigned



DATA TYPES

Unsigned char (cont')

Write an 8051 C program to send values 00 - FF to port P1.

Solution:

```
#include <reg51.h>
void main(void)
{
    unsigned char z;
    for
        (z=0; z<=255; z++)
    } P1=z;
```

1. Pay careful attention to the size of the data
2. Try to use unsigned char instead of int if possible

Write an 8051 C program to send hex values for ASCII characters of 0, 1, 2, 3, 4, 5, A, B, C, and D to port P1.

Solution:

```
#include <reg51.h>
void main(void) {

    unsigned char mynum[]="012345ABCD";
    unsigned char z;
    for
        (z=0; z<=10; z++)
    } P1=mynum[z];
```



DATA TYPES

Unsigned char (cont')

Write an 8051 C program to toggle all the bits of P1 continuously.

Solution:

```
//Toggle P1 forever
#include <reg51.h>
void
main(void) {
    for
        (;;) {
            p1=0x55;
            p1=0xAA; }
}
```



DATA
TYPES
Signed char

The signed char is an 8-bit data type

- Use the MSB D7 to represent - or +
- Give us values from -128 to +127

We should stick with the unsigned char unless the data needs to be represented as signed numbers

- temperature

Write an 8051 C program to send values of -4 to +4 to port P1.

Solution:

```
//Signed numbers
#include <reg51.h>
void main(void) {
    char mynum[]={+1,-1,+2,-2,+3,-3,+4,-4};
    unsigned char z;
    for
    (z=0;z<=8;z++)
    } P1=mynum[z];
```



DATA TYPES

Unsigned and Signed int

The unsigned int is a 16-bit data type

- Takes a value in the range of 0 to 65535 (0000 - FFFFH)
- Define 16-bit variables such as memory addresses
- Set counter values of more than 256 •

Since registers and memory accesses are in 8-bit chunks, the misuse of int variables will result in a larger hex file

Signed int is a 16-bit data type

- Use the MSB D15 to represent - or + • We have 15 bits for the magnitude of the number from -32768 to +32767



DATA TYPES

Single Bit (cont')

Write an 8051 C program to toggle bit D0 of the port P1 (P1.0) 50,000 times.

Solution:s

`#include <reg51.h>` *allows access to the single bits of the SFR registers*
`sbit MYBIT=P1^0;`

```
void  
main(void) {  
    unsigned int z;  
    for  
    (z=0;z<=50000;z++) {  
        MYBIT=0;  
        MYBIT=1; }  
}
```



DATA TYPES

Bit and sfr

The bit data type allows access to single bits of bit-addressable memory spaces 20 - 2FH

To access the byte-size SFR registers, we use the sfr data type

Data Type	Size in Bits	Data Range/Usage
unsigned char	8-bit	0 to 255
(signed) char	8-bit	-128 to +127
unsigned int	16-bit	0 to 65535
(signed) int	16-bit	-32768 to +32767
sbit	1-bit	SFR bit-addressable only
bit	1-bit	RAM bit-addressable only
sfr	8-bit	RAM addresses 80 - FFH only



Statements

- Assignment statement:

variable = *constant* or *expression* or *variable*

examples: *upper* = 60;

I = *I* + 5;

J = *I*;

Operators

Arithmetic Operators

- Arithmetic: +, -, *, /
- Relational comparisons: >, >=, <, <=
- Equality comparisons: ==, !=
- Logical operators: && (and), || (or), ^ (XOR)
- Increment and decrement: ++, --
- Example:

```
if (x != y) && (c == b)
{
    a=c + d*b;
    a++;
}
```

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B) is true.

Operator	Description	Example
+	Adds two operands.	A + B = 30
-	Subtracts second operand from the first.	A - B = -10
*	Multiplies both operands.	A * B = 200
/	Divides numerator by de-numerator.	B / A = 2
%	Modulus Operator and remainder of after an integer division.	B % A = 0
++	Increment operator increases the integer value by one.	A++ = 11
--	Decrement operator decreases the integer value by one.	A-- = 9

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.	(A <= B) is true.

Example – Adder program (add 2 16-bit numbers)

```
$INCLUDE (C8051F020.inc)
XL equ 0x78
XH equ 0x79
YL equ 0x7A
YH equ 0x7B
    cseg at 0
    ljmp Main
cseg at 100h
; Disable watchdog timer
Main:  mov 0xFF, #0DEh
       mov 0xFF, #0ADh
       mov a, XL
       add a, YL
       mov XL, a
       mov a, XH
       addc a, YH
       mov XH, a
       nop
end
```

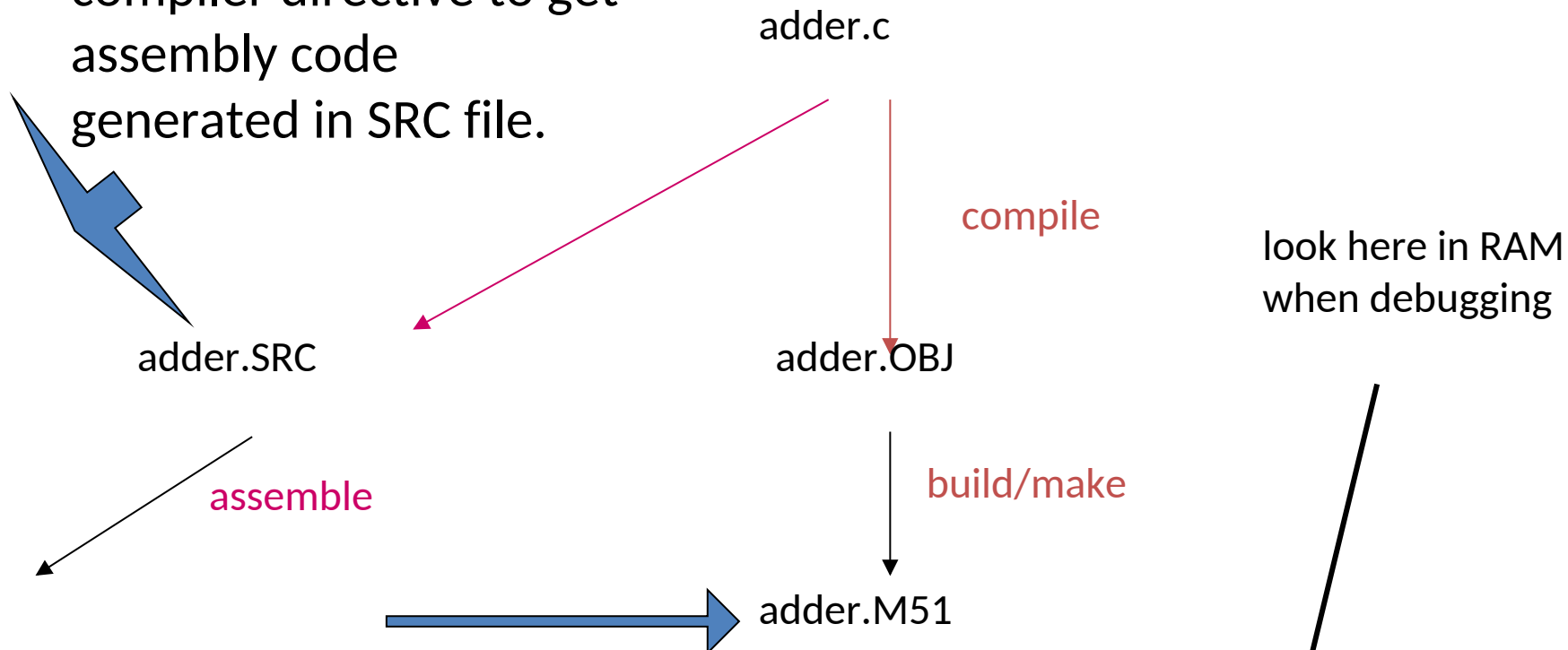
```
#include <c8051f020.h>
void main (void) {
    int x, y, z; //16-bit variables
    // disable watchdog timer
    WDTCN = 0xde;
    WDTCN = 0xad;
    z = x + y;
}
```

The assembly version

The C version

Compilation Process (Keil)

Use the #pragma CODE
compiler directive to get
assembly code
generated in SRC file.



Map file shows where variables
are stored. One map file is
generated per project.

look here in RAM
when debugging

Symbol Table in M51 file:

```
----- DO
D:0008H    SYMBOL    x
D:000AH    SYMBOL    y
D:000CH    SYMBOL    z
----- ENDDO
```

adder.SRC

```
x?040:    DS    2
y?041:    DS    2
z?042:    DS    2
```

main:

```
                                ; SOURCE LINE # 12
; int x, y, z;
;     WDTCN = 0xde;             // disable watchdog timer
                                ; SOURCE LINE # 14
    MOV     WDTCN,#0DEH
;     WDTCN = 0xad;
                                ; SOURCE LINE # 15
    MOV     WDTCN,#0ADH
;     z = x + y;
                                ; SOURCE LINE # 17
    MOV     A,x?040+01H
    ADD     A,y?041+01H
    MOV     z?042+01H,A
    MOV     A,x?040
    ADDC    A,y?041
    MOV     z?042,A
;     }                        ; SOURCE LINE # 18
    RET
; END OF main
    END
```

Bitwise Logic Instructions

- AND & $n = n \& 0xF0;$
- OR |
- XOR ^
- left shift << $n = n \& (0xFF \ll 4)$
- right shift >> $n = n \& \sim(0xFF \gg 4)$
- 1's complement ~

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	$(A \& B) = 12$, i.e., 0000 1100
	Binary OR Operator copies a bit if it exists in either operand.	$(A B) = 61$, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	$(A \wedge B) = 49$, i.e., 0011 0001
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	$(\sim A) = -60$, i.e., 1100 0100 in 2's complement form.
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	$A \ll 2 = 240$ i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	$A \gg 2 = 15$ i.e., 0000 1111

```
#include <stdio.h>
```

```
main() {
```

```
    unsigned int a = 60; /* 60 = 0011 1100 */
    unsigned int b = 13; /* 13 = 0000 1101 */
    int c = 0;
```

```
    c = a & b;      /* 12 = 0000 1100 */
    printf("Line 1 - Value of c is %d\n", c );
```

```
    c = a | b;      /* 61 = 0011 1101 */
    printf("Line 2 - Value of c is %d\n", c );
```

```
    c = a ^ b;      /* 49 = 0011 0001 */
    printf("Line 3 - Value of c is %d\n", c );
```

```
    c = ~a;         /* -61 = 1100 0011 */
    printf("Line 4 - Value of c is %d\n", c );
```

```
    c = a << 2;     /* 240 = 1111 0000 */
    printf("Line 5 - Value of c is %d\n", c );
```

```
    c = a >> 2;     /* 15 = 0000 1111 */
    printf("Line 6 - Value of c is %d\n", c );
```

```
}
```

Assignment Operators

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	C = A + B will assign the value of A + B to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
=	Bitwise inclusive OR and assignment operator.	C = 2 is same as C = C 2

```
#include <stdio.h>
```

```
main() {
```

```
    int a = 21;
```

```
    int c ;
```

```
    c = a;
```

```
    printf("Line 1 - = Operator Example, Value of c = %d\n", c );
```

```
    c += a;
```

```
    printf("Line 2 - += Operator Example, Value of c = %d\n", c );
```

```
    c -= a;
```

```
    printf("Line 3 - -= Operator Example, Value of c = %d\n", c );
```

```
    c *= a;
```

```
    printf("Line 4 - *= Operator Example, Value of c = %d\n", c );
```

```
    c /= a;
```

```
    printf("Line 5 - /= Operator Example, Value of c = %d\n", c );
```

```
    c = 200;
```

```
    c %= a;
```

```
    printf("Line 6 - %= Operator Example, Value of c = %d\n", c );
```

```
    c <<= 2;
```

```
    printf("Line 7 - <<= Operator Example, Value of c = %d\n", c );
```

```
    c >>= 2;
```

```
    printf("Line 8 - >>= Operator Example, Value of c = %d\n", c );
```

```
    c &= 2;
```

```
    printf("Line 9 - &= Operator Example, Value of c = %d\n", c );
```

```
    c ^= 2;
```

```
    printf("Line 10 - ^= Operator Example, Value of c = %d\n", c );
```

```
    c |= 2;
```

```
    printf("Line 11 - |= Operator Example, Value of c = %d\n", c );
```

```
}
```

https://www.tutorialspoint.com/cprogramming/c_assignment_operators.htm

C- Operators

Misc Operators

Operator	Description	Example
sizeof()	Returns the size of a variable.	sizeof(a), where a is integer, will return 4.
&	Returns the address of a variable.	&a; returns the actual address of the variable.
*	Pointer to a variable.	*a;
? :	Conditional Expression.	If Condition is true ? then value X : otherwise value Y

```
#include <stdio.h>
```

```
main() {
```

```
    int a = 4;  
    short b;  
    double c;  
    int* ptr;
```

```
    /* example of sizeof operator */
```

```
    printf("Line 1 - Size of variable a = %d\n", sizeof(a) );  
    printf("Line 2 - Size of variable b = %d\n", sizeof(b) );  
    printf("Line 3 - Size of variable c= %d\n", sizeof(c) );
```

```
    /* example of & and * operators */
```

```
    ptr = &a; /* 'ptr' now contains the address of 'a' */  
    printf("value of a is %d\n", a);  
    printf("**ptr is %d.\n", *ptr);
```

```
    /* example of ternary operator */
```

```
    a = 10;  
    b = (a == 1) ? 20: 30;  
    printf( "Value of b is %d\n", b );
```

```
    b = (a == 10) ? 20: 30;  
    printf( "Value of b is %d\n", b );
```

```
}
```

Line 1 - Size of variable a = 4
Line 2 - Size of variable b = 2
Line 3 - Size of variable c= 8
Value of a is 4 *ptr is 4.
Value of b is 30
Value of b is 20

Example – Logic in Assembly and C

Main:

```
mov WDTCN, #0DEh
mov WDTCN, #0ADh
xrl a, #0xF0    ; invert bits 7-4
orl a, #0x0C    ; set bits 3-2
anl a, #0xFC    ; reset bits 1-0
mov P0, a      ; send to port0
```

```
void main (void) {
    char x;
    WDTCN = 0xDE;
    WDTCN = 0xAD;
    x = x ^ 0xF0;
    x = x | 0x0C;
    x = x & 0xFC;
    P0 = x;
}
```

Loop Statements - While

- While loop:

while (condition) { statements }

while condition is true, execute statements

if there is only one statement, we can lose the {}

Example: `while (1) ;            // loop forever`

Loop Statements - For

- For statement:

for (initialization; condition; increment) {statements}

initialization done before statement is executed

condition is tested, if true, execute statements
do *increment* step and go back and test condition again

repeat last two steps until condition is not true

Example: for loop

```
for (n = 0; n < 1000; n++)
```

n++ means $n = n + 1$

Be careful with signed integers!

```
for (i=0; i < 33000; i++) LED = ~LED;
```

Why is this an infinite loop?

Loops: do - while

```
do  
    statements  
while (expression);
```

Test made at the bottom of the loop

Decision – if statement

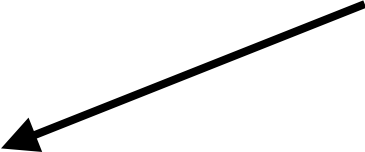
```
if (condition1)  
    {statements1}  
else if (condition2)  
    {statements2}  
...  
else  
    {statementsn}
```

Decision – switch statement

```
switch (expression) {  
    case const-expr: statements  
    case const-expr: statements  
    default: statements  
}
```

Example: switch

```
switch (unibble) {  
    case 0x00 : return (0xC0);  
    case 0x01 : return (0xF9);  
    case 0x02 : return (0xA4);  
    case 0x03 : return (0xC0);  
    default : return (0xFF);  
}
```

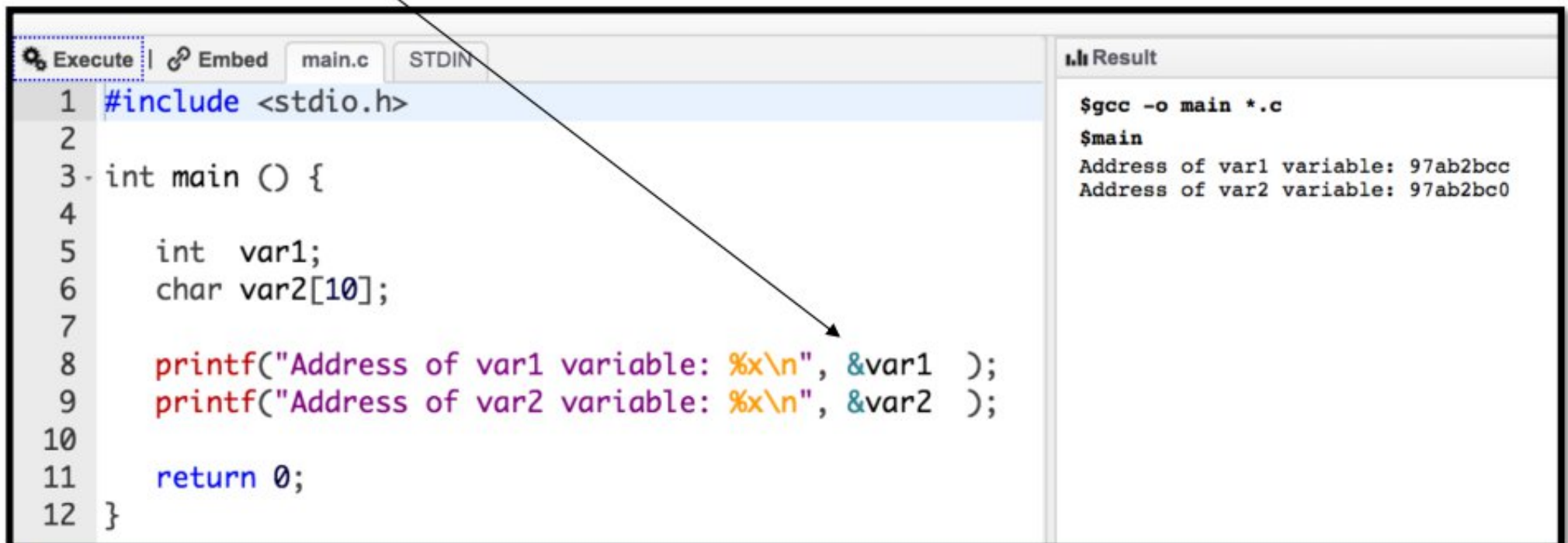


Need a statement like “return”
or “break” or execution falls
through to the next case (unlike
VHDL)

C-Pointers

Every variable is a memory location and every memory location has its address defined which can be accessed using ampersand (**&**) operator, which denotes an address in memory.

A **pointer** is a variable whose value is the address of another variable, i.e., direct address of the memory location.



The screenshot shows a C program being executed in an IDE. The code defines two variables, `var1` (an integer) and `var2` (a character array of size 10). It then uses `printf` to print their memory addresses. The output shows the address of `var1` as `97ab2bcc` and the address of `var2` as `97ab2bc0`. An arrow points from the text 'denotes an address in memory' to the `&var1` and `&var2` expressions in the code.

```
1 #include <stdio.h>
2
3 int main () {
4
5     int var1;
6     char var2[10];
7
8     printf("Address of var1 variable: %x\n", &var1 );
9     printf("Address of var2 variable: %x\n", &var2 );
10
11     return 0;
12 }
```

Result

```
$gcc -o main *.c
$main
Address of var1 variable: 97ab2bcc
Address of var2 variable: 97ab2bc0
```

Using Pointers

Difference between VAR_Name, & VAR_Name, *VAR_Name,

Execute | Embed | main.c | STDIN

```
1 #include <stdio.h>
2
3 int main () {
4
5     int var1=6;
6     char var2=4;
7
8     int *ip; /* pointer to an integer */
9     double *dp; /* pointer to a double */
10    float *fp; /* pointer to a float */
11    char *ch; /* pointer to a character */
12
13    ip = &var1; /* store address of var in pointer variable*/
14
15    printf("Address of var1 variable: %x\n", &var1 );
16    printf("Address of var2 variable: %x\n", &var2 );
17
18    /* address stored in pointer variable */
19    printf("Address stored in ip (var1) variable: %x\n", ip );
20
21    /* access the value using the pointer */
22    printf("Value of *ip variable: %d\n", *ip );
23
24    printf("Address of ip variable: %x\n", &ip );
25    return 0;
26 }
```

Result

```
$gcc -o main *.c
$main
Address of var1 variable: 66abce7c
Address of var2 variable: 66abce7b
Address stored in ip (var1) variable: 66abce7c
Value of *ip variable: 6
Address of ip variable: 66abce70
```

pointer variable declaration

IP=&VAR1=66abce7c

&VAR2=66abce7b

&IP=66abce70

VAR1=6
VAR2=4
IP=66abce7c

MEMORY

*IP=6

Pointer Concepts

Sr.No.	Concept & Description
1	Pointer arithmetic  There are four arithmetic operators that can be used in pointers: ++, --, +, -
2	Array of pointers  You can define arrays to hold a number of pointers.
3	Pointer to pointer  C allows you to have pointer on a pointer and so on.
4	Passing pointers to functions in C  Passing an argument by reference or by address enable the passed argument to be changed in the calling function by the called function.
5	Return pointer from functions in C  C allows a function to return a pointer to the local variable, static variable, and dynamically allocated memory as well.

Examples:

***p++** // same as ***(p++)**: increment pointer, and dereference unincremented address
***++p** // same as ***(++p)**: increment pointer, and dereference incremented address
++*p // same as **++(*p)**: dereference pointer, and increment the value it points to
(*p)++ // dereference pointer, and post-increment the value it points to

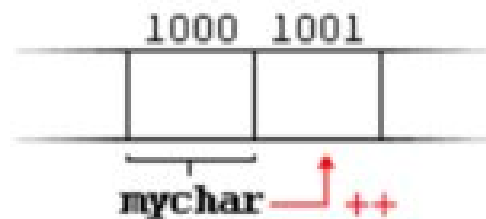
Read more about pointer:

<http://www.cplusplus.com/doc/tutorial/pointers/>

Pointers and Data Types

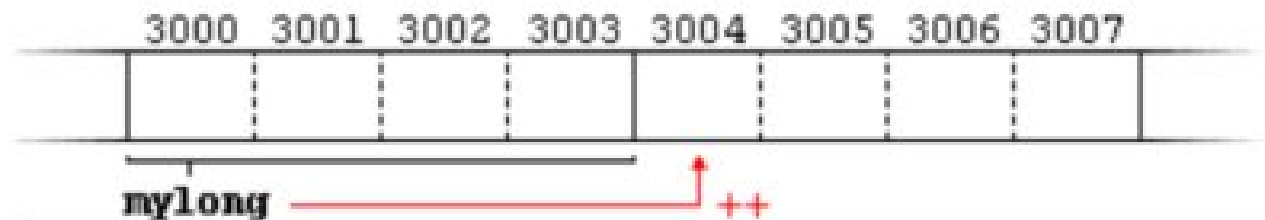
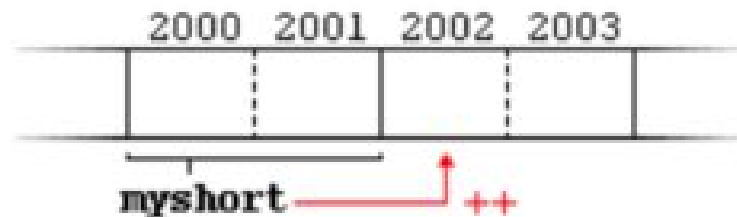
Pointer Data Types

```
1 char *mychar;  
2 short *myshort;  
3 long *mylong;
```



Pointer Increments

```
1 ++mychar;  
2 ++myshort;  
3 ++mylong;
```



Pointer Example

```
Execute | Embed main.c STDIN
1 // more pointers
2 #include <stdio.h>
3 int main ()
4 {
5     int numbers[5];
6     int * p;
7     int n;
8     p = numbers; * p = 10;
9     p++; *p = 20;
10    p = &numbers[2]; *p = 30;
11    p = numbers + 3; *p = 40;
12    p = numbers; *(p+4) = 50;
13    for (n=0; n<5; n++){
14        printf("%d ", numbers[n]);
15    }
16    return 0;
17 }
```

Result

```
$gcc -o main *.c
$main
10 , 20 , 30 , 40 , 50 ,
```

66abce74

66abce72

66abce70

Number[4]=50
Number[3]=40
Number[2]=30
Number[1]=20
Number[0]=10

MEMORY

← Pointer p
*p=20

Pointer Function with Argument

```
Execute | Embed main.c STDIN
1 // pointers as arguments:
2 #include <stdio.h>
3
4
5 void increment_all (int* start, int* stop)
6 {
7     int * current = start;
8     printf("Start & Stop Address: %X, %X , \n",start, stop);
9     while (current != stop) {
10         ++(*current); // increment value pointed
11         ++current;    // increment pointer
12     }
13 }
14
15 void print_all (const int* start, const int* stop)
16 {
17     const int * current = start;
18     while (current != stop) {
19         printf("%d , \n",*current);
20         ++current;    // increment pointer
21     }
22 }
23
24 int main ()
25 {
26     int numbers[ ] = {10,20,30};
27     printf("Addresses %X, %X, %X , \n",numbers, numbers+1, numbers+2);
28     increment_all (numbers,numbers+3);
29     print_all (numbers,numbers+3);
30     return 0;
31 }
```

Result

```
$gcc -o main *.c
$main
Addresses CC4E7100, CC4E7104, CC4E7108 ,
Start & Stop Address CC4E7100, CC4E710C ,
11 ,
21 ,
31 ,
```

CC4E710C

CC4E7108 Number[2]

CC4E7104 Number[1]

CC4E7100 Number[0]

MEMORY

*current
←

Structure Array in C

```
struct Position {  
    char x;  
    char y;  
    char z;  
    int time;  
    short scale;  
} array[2];
```

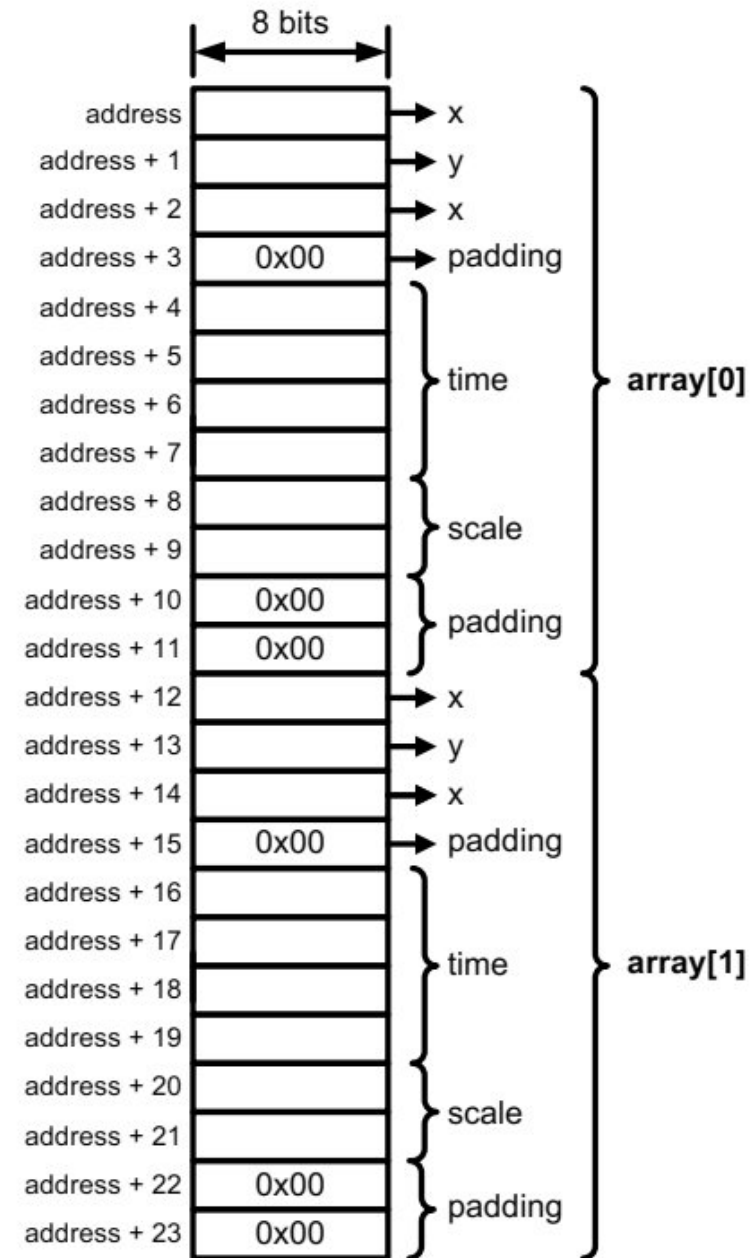


```
struct Position {  
    char x;  
    char y;  
    char z;  
    char padding_1[1];  
    int time;  
    short scale;  
    char padding_2[2];  
} array[2];
```

A data Structure in C aggregates multiple basic variables in a single structure that are aligned To their required memory boundaries.

Here we define a structure array.

Structure arrays can be packed structures



__unpacked Data Structure

Structure arrays can be packed structures – using in USB communications

The screenshot displays an IDE with two main panels. The left panel, titled 'all Stack + Locals', shows a tree view of the program's memory layout. The right panel shows the C source code for 'Combine_Assem_C_Code_2.c'.

Memory Layout (all Stack + Locals):

Name	Location/Value
main	0x08000338
array	0x20000648
array[0]	0x20000648
array[0].a1	0x60
array[0].b1	0x60200000
array[0].c1	0x0002
array[0].d1	0x00026020
array[0].e1	0x20
array[1]	0x20000654
array[1].a1	0x0C
array[1].b1	0x0000000D
array[1].c1	0x000E
array[1].d1	0x0000000F
array[1].e1	0x10

C Source Code (Combine_Assem_C_Code_2.c):

```
19 }
20 return t;
21 }
22
23 int main(void) {
24     __packed struct Position {
25         uint8_t a1;
26         uint32_t b1;
27         uint16_t c1;
28         uint32_t d1;
29         uint8_t e1;
30     } array[2];
31
32     // static int32_t s = 0;
33     // s= sum5(2, 2, 3, 6);
34     array[1].a1=12;
35     array[1].b1=13;
36     array[1].c1=14;
37     array[1].d1=15;
38     array[1].e1=16;
39     while(1);
40 }
```

Number Formats

Using printf() Function

```
#include<stdio.h>
int x = 0x12;
main()
{

    printf("The color: %s\n", "blue");
    printf("First number: %d\n", 12345);
    printf("Second number: %04d\n", 25);
    printf("Third number?: %i\n", 1234);
    printf("Float number: %3.2f\n", 3.14159);
    printf("Hexadecimal: %x\n", 255);
    printf("Hexadecimal-----: %d\n", x);
    printf("Octal: %o\n", 255);
    printf("Unsigned value: %u\n", 150);
    printf("Just print the percentage sign %%\n", 10);

}
```

What are the result?

Function Data Type & Function Declaration

// function declaration

```
int func();
```

```
int main() {
```

// function call

```
    int i = func();
```

```
}
```

// function definition

```
int func() {
```

```
    return 0;
```

```
}
```

For function declaration we can provide a function name at the time of its declaration and its **actual definition** can be given anywhere else.

Note the function type is INT

Literal Constants

- A literal is a constant, but a constant is not a literal
 - `#define MAXINT 32767`
 - `const int MAXINT = 32767;`
 - **Constants** are labels that represent a literal
 - **Literals** are values, often assigned to symbolic constants and variables
- Literals or Literal Constants
 - Are "hard coded" values
 - May be numbers, characters, or strings
 - May be represented in a number of formats (decimal, hexadecimal, binary, character, etc.)
 - Always represent the **same value** (5 always represents the quantity five)

Defining Constants - Example

There are two simple ways in C to define constants –

- Using **#define** preprocessor.
- Using **const** keyword.

```
#include <stdio.h>

#define LENGTH 10
#define WIDTH 5
#define NEWLINE '\n'

int main() {
    int area;

    area = LENGTH * WIDTH;
    printf("value of area : %d", area);
    printf("%c", NEWLINE);

    return 0;
}
```

```
#include <stdio.h>

int main() {
    const int LENGTH = 10;
    const int WIDTH = 5;
    const char NEWLINE = '\n';
    int area;

    area = LENGTH * WIDTH;
    printf("value of area : %d", area);
    printf("%c", NEWLINE);

    return 0;
}
```

Note that it is a good programming practice to define constants in CAPITALS.

Integer Literal

An **integer literal** can be a decimal, octal, or hexadecimal constant.

A prefix specifies the base or radix:

- 0x or 0X for hexadecimal,
- 0 for octal,
- and nothing for decimal.

An integer literal can also have a suffix that is a combination of U and L, for unsigned and long, respectively. The suffix can be uppercase or lowercase and can be in any order.

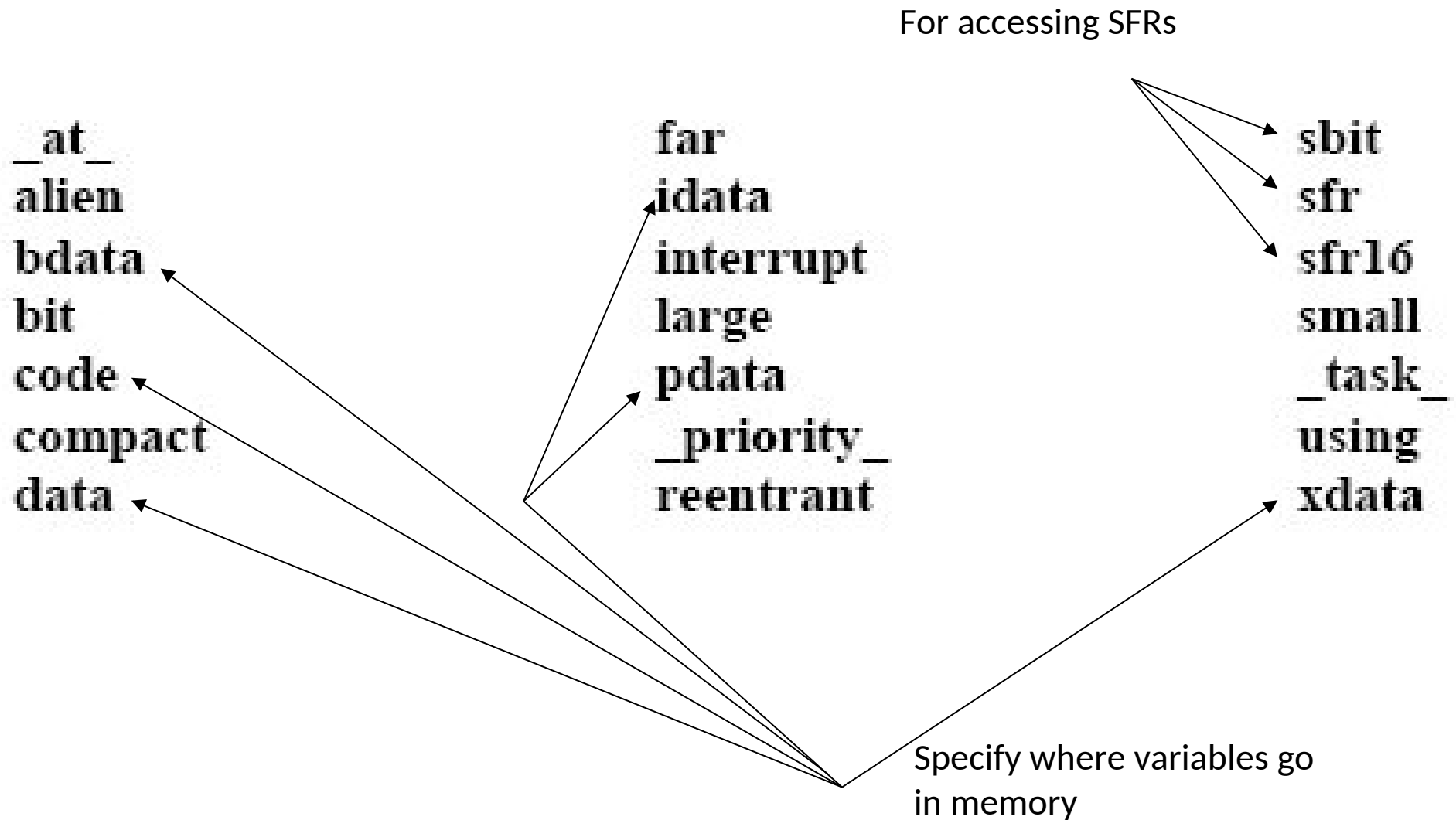
```
85      /* decimal */
0213    /* octal */
0x4b    /* hexadecimal */
30      /* int */
30u     /* unsigned int */
30l     /* long */
30ul    /* unsigned long */
```

```
#include <stdio.h>
#include <limits.h>
int main()
{
    int d = 42;
    int o = 051;
    int x = 0x2b;
    int X = 0X2A;
    int b = 0b101010; // C++14

    printf("%d, %d, %d, %d, %d", d, o, x, X, b);
    /* Output: 42, 41, 43, 42, 42 */

    return 0;
}
```

C Extensions: Additional Keywords



Accessing Specific Memory

Memory Type	Description
→ code	Program memory (64 KBytes); accessed by opcode MOVC @A+DPTR.
→ data	Directly addressable internal data memory; fastest access to variables (128 bytes).
→ idata	Indirectly addressable internal data memory; accessed across the full internal address space (256 bytes).
→ bdata	Bit-addressable internal data memory; supports mixed bit and byte access (16 bytes).
→ xdata	External data memory (64 KBytes); accessed by opcode MOVX @DPTR.
far	Extended RAM and ROM memory spaces (up to 16MB); accessed by user defined routines or specific chip extensions (Philips 80C51MX, Dallas 390).
pdata	Paged (256 bytes) external data memory; accessed by opcode MOVX @Rn.

type	read	write	register
direct	MOV Rn,direct MOV direct,direct	MOV direct,Rn MOV direct,@DPTR MOV direct,#data	R0~R7,A
indirect	MOV direct,@Ri MOVX A,@DPTR	MOV @Ri,direct MOVX @DPTR,A MOV @Ri,#data	R0,R1,DPTR,A
index direct	MOV A,#30h MOV DPTR,#300H MOVC A,@A+DPTR	X	A,DPTR

Location of variable	Assembly code	Assembly code size*
register Rn	INC Rn	1
data	INC var	2
idata	MOV R0,#LOW var INC @R0	3
pdata	MOV R0,#LOW var MOVX A,@R0 INC A MOVX @R0,A	5
xdata	MOV DPTR,#var MOVX A,@DPTR INC A MOVX @DPTR,A	6

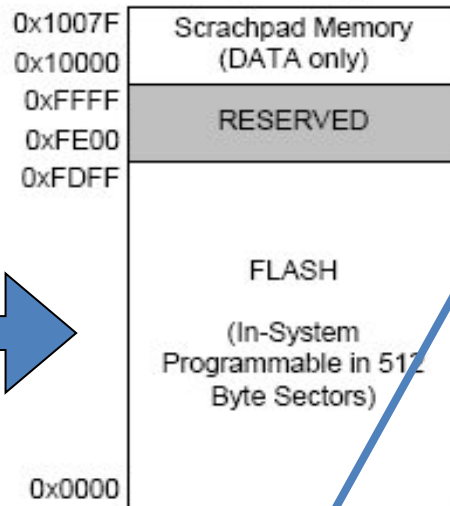
* 8051's instructions are 1-3 bytes long

C Access to 8051 Memory

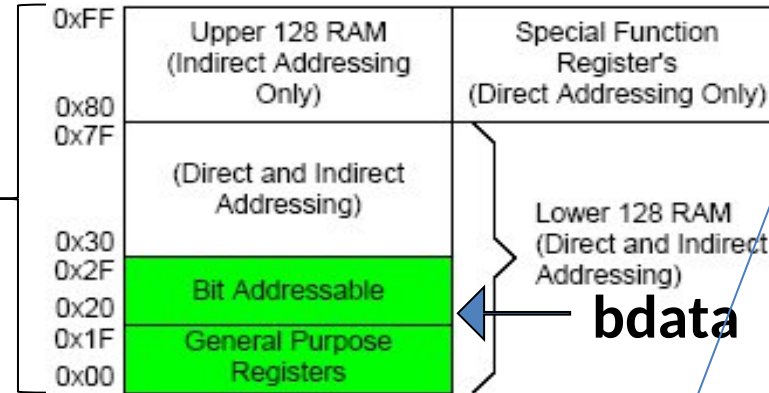
Above 80H the special function registers are located, which are again directly addressable

code: program memory accessed by `movc @a + dptr` (Index indirect)

PROGRAM/DATA MEMORY (FLASH)



DATA MEMORY (RAM) INTERNAL DATA ADDRESS SPACE



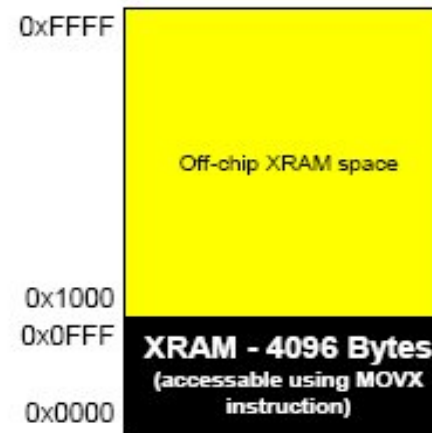
SFR

data

idata

It is directly addressable, so that instructions like 'MOV A,x' are usable
 Add: 0x7F-0x00H
 For data, indirect access is also possible `MOV A,@Ri`

EXTERNAL DATA ADDRESS SPACE



xdata

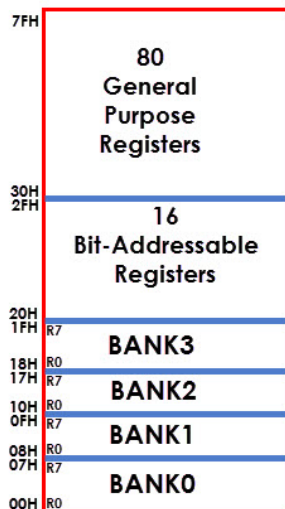
`MOVX @DPTR`

`movc @a + dptr`

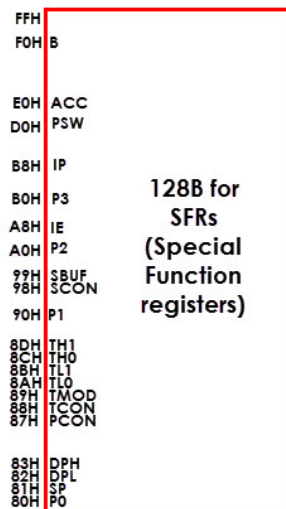
data

SFR

idata



Lower 128B (00H - 7FH)
 (Direct and Indirect Addressing)



Upper 128B (80H - FFH)
 (Direct Addressing)



(Indirect Addressing)

FFFh

0FFh

07Fh

000h

DATA

SFR

IDATA

PDATA

XDATA

CODE

`MOV A,@Ri`

`MOVX @Rn`

Best & Worst for Memory Variables

DATA: 128 bytes; SMALL model default location

- Best For:

Frequently accessed data requiring the fastest access. Interrupt routines whose run time is critical should use DATA, usually by declaring the function as "SMALL". Another recommended usage is for background code that is frequently run and has many parameters to pass. If you are using re-entrant functions, the re-entrant stacks should be located here as a priority.

- Worst For: Variable arrays and structures that contain more than a few bytes.

IDATA: 128 bytes or 256 bytes; Not model-dependant

- Best For:

Fast access to data arrays and structures of limited size (up to around 32 bytes each) but not totalling more than 64 or so bytes. As these data types require indirect addressing, they are ideally placed in the indirectly addressable area. The stack is also located in IDATA as it is indirectly addressed.

- Worst For: Large data arrays and/or fast access words.

CODE: 64K bytes

- Best For: Constants and large lookup tables, plus opcodes, of course!

- Worst For: Variables! It's ROM - Read Only Memory that can not be written to.

PDATA: 256 bytes; COMPACT model default area

- Best For:

Medium speed interrupt and fast background char (8 bit) variables and moderate-sized arrays and structures **EXTERNAL**. Also good for variables which need to be viewed in real-time using an in-circuit emulator with dual ported memory.

- Worst For:

Very large data arrays and structure above 256 bytes.

Very frequently used data (in interrupts etc..).

Integer and long data.

XDATA: up to 64K bytes; LARGE model default area

- Best For:

Large variable arrays and structures (over 256 bytes)

Slow or infrequently-used background variables. Also good for variables which need to be viewed in real-time using an in-circuit emulator with dual ported memory.

- Worst For:

Frequently-accessed or fast interrupt variable

8051 Programming Examples

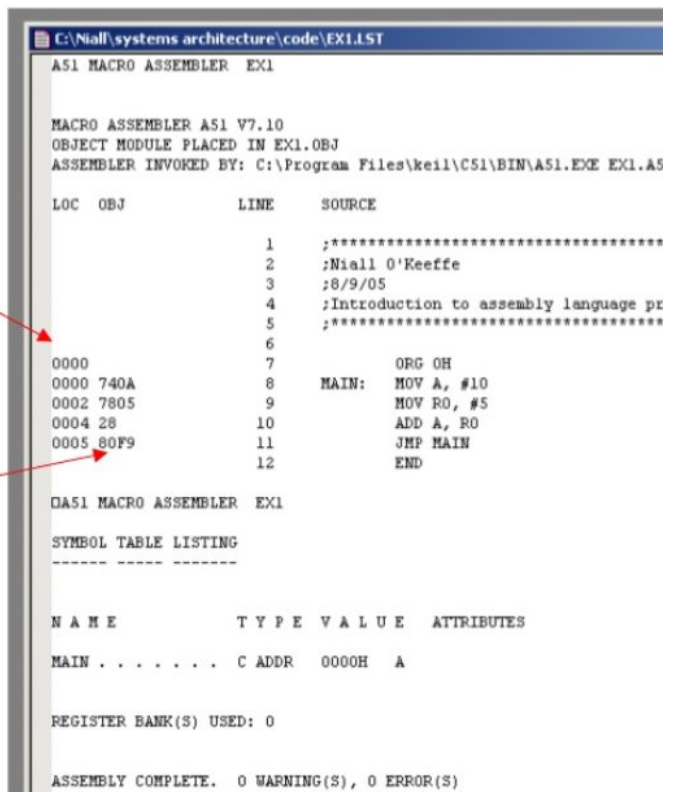
- C program example to add 2 numbers

```
void main()
{
    unsigned char x=5,y=6,z;
    z = x + y;
}
```

- Same code written using assembly language

```
MOV    A,#05H
ADD    A,#06H
MOV    R0,A           ;result stored in R0
```

Only registers used.
No memory used



Program Memory Address

Machine code

```
C:\Niall\systems architecture\code\EX1.A5
A51 MACRO ASSEMBLER EX1

MACRO ASSEMBLER A51 V7.10
OBJECT MODULE PLACED IN EX1.OBJ
ASSEMBLER INVOKED BY: C:\Program Files\keil\C51\BIN\A51.EXE EX1.A5

LOC  OBJ          LINE    SOURCE
1      ;*****
2      ;Niall O'Keeffe
3      ;8/9/05
4      ;Introduction to assembly language pr
5      ;*****
6
7      0000          7      ORG 0H
8      0000 740A      8      MAIN: MOV A, #10
9      0002 7805      9      MOV R0, #5
10     0004 28        10     ADD A, R0
11     0005 80F9      11     JMP MAIN
12                                END

DAS1 MACRO ASSEMBLER EX1

SYMBOL TABLE LISTING
-----

NAME                TYPE VALUE  ATTRIBUTES
MAIN . . . . . C ADDR 0000H  A

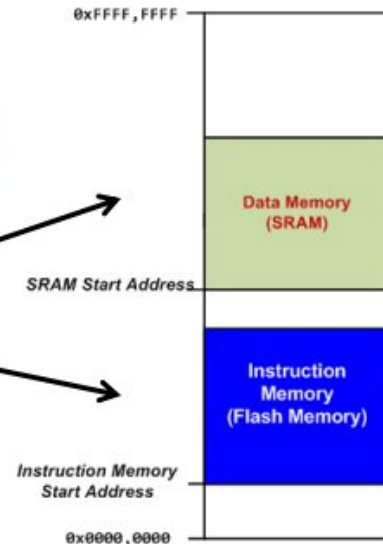
REGISTER BANK(S) USED: 0

ASSEMBLY COMPLETE. 0 WARNING(S), 0 ERROR(S)
```

Loading Code and Data into Memory

```
int counter;  
  
int a[5] = {1, 2, 3, 4, 5};  
  
int main(void){  
    int i;  
  
    int b[5];  
  
    counter = 0;  
    for (i = 0; i < 5; i++){  
        b[i] = a[i];  
        counter++  
    }  
    while(1);  
}
```

Dissection of a C Program:
Copying an Array

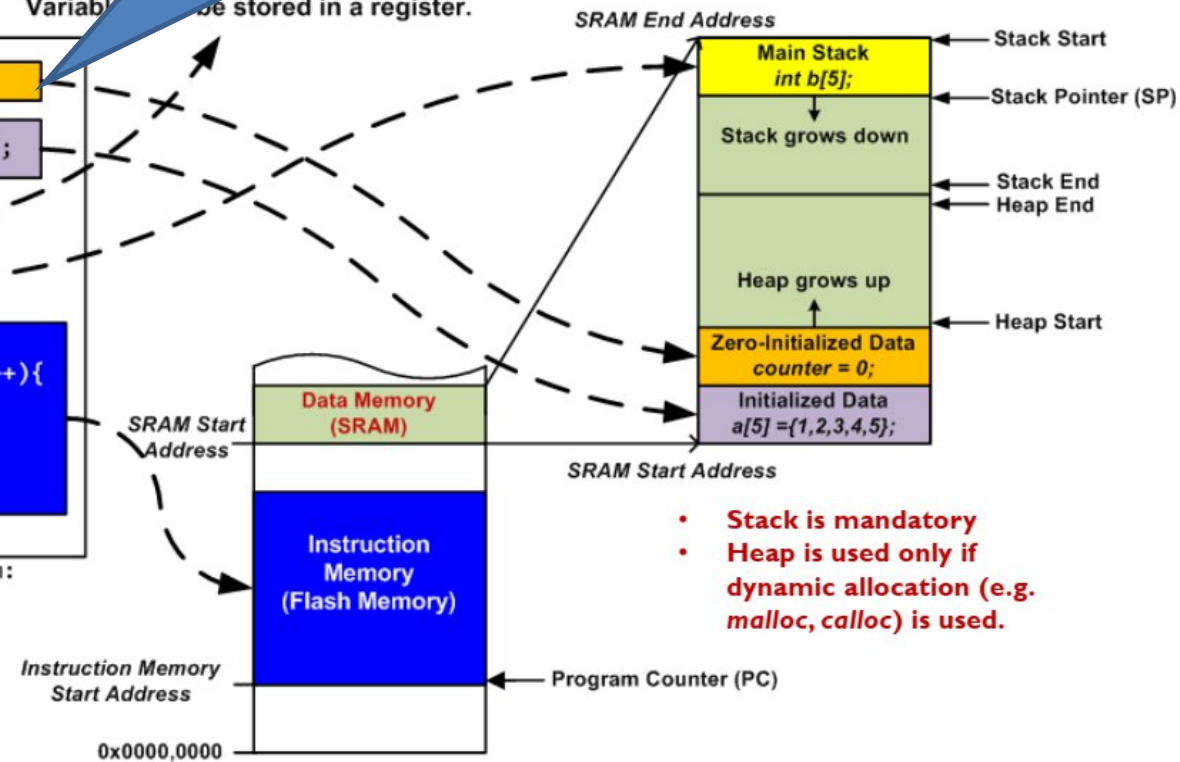


After reset, Startup code will **copy and initialize variables from Flash to RAM**, initialize map and peripherals (enable/disable EEPROM/Flash, set register, disable/enable watchdog, ...). Reset vector typically points to `_Startup()` function which runs before your Main program.

```
int counter;  
  
int a[5] = {1, 2, 3, 4, 5};  
  
int main(void){  
    int i;  
  
    int b[5];  
  
    counter = 0;  
    for (i = 0; i < 5; i++){  
        b[i] = a[i];  
        counter++  
    }  
    while(1);  
}
```

Dissection of a C Program:
Copying an Array

To improve performance, some variables are stored in memory. Variables that are frequently used are stored in a register.



- **Stack is mandatory**
- **Heap is used only if dynamic allocation (e.g. `malloc`, `calloc`) is used.**

C Extensions for 8051 (Cygnal)

- New data types:

Example:

bit	bit new_flag;	//stored in 20-2F
sbit	sbit LED = P1^6;	
sfr	sfr SP = 0x81;	//stack pointer
sfr16	sfr16 DP = 0x82;	// data pointer

\$INCLUDE ([c8051F020.h](#))

C Data Types With Extensions

Data Types	Bits	Bytes	Value Range
bit †	1		0 to 1
signed char	8	1	-128 to +127
unsigned char	8	1	0 to 255
enum	8 / 16	1 or 2	-128 to +127 or -32768 to +32767
signed short	16	2	-32768 to +32767
unsigned short	16	2	0 to 65535
signed int	16	2	-32768 to +32767
unsigned int	16	2	0 to 65535
signed long	32	4	-2147483648 to +2147483647
unsigned long	32	4	0 to 4294967295
float	32	4	$\pm 1.175494\text{E-}38$ to $\pm 3.402823\text{E}+38$
sbit †	1		0 or 1
sfr †	8	1	0 to 255
sfr16 †	16	2	0 to 65535

Declaring Variables in Memory

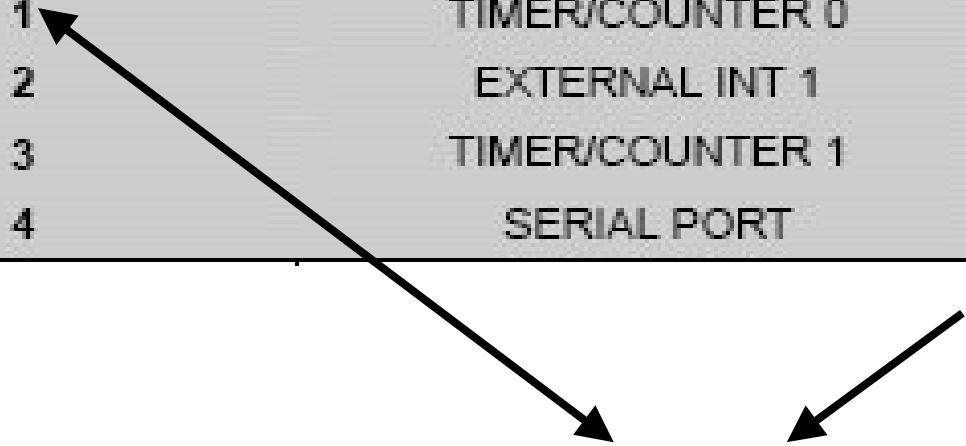
```
char data temp;  
char xdata temp;  
char idata varx;  
int xdata array[100];  
char code text[] = "Enter data";
```

Example: Accessing External Memory

- Program defines two 256 element arrays in external memory
- First array is filled with values that increase by 2 each location.
- First array is copied to second array.
- Similar to block move exercise done in assembly.
- [xdata_move.c](#)

Interrupts – Original 8051

Interrupt Number	Interrupt Description	Address
0	EXTERNAL INT 0	0003h
1	TIMER/COUNTER 0	000Bh
2	EXTERNAL INT 1	0013h
3	TIMER/COUNTER 1	001Bh
4	SERIAL PORT	0023h



```
void timer0 (void) interrupt 1 using 2 {  
    if (++interruptcnt == 4000) {  
        second++;  
        interruptcnt = 0;  
    }  
}
```

Specify register bank 2

```
/* count to 4000 */  
/* second counter */  
/* clear int counter */
```

Other Interrupt Numbers

Interrupt Number	Address
0	0003h
1	000Bh
2	0013h
3	001Bh
4	0023h
5	002Bh
6	0033h
7	003Bh
8	0043h
9	004Bh
10	0053h
11	005Bh
12	0063h
13	006Bh
14	0073h
15	007Bh

Interrupt Number	Address
16	0083h
17	008Bh
18	0093h
19	009Bh
20	00A3h
21	00ABh
22	00B3h
23	00BBh
24	00C3h
25	00CBh
26	00D3h
27	00DBh
28	00E3h
29	00EBh
30	00F3h
31	00FBh

Interrupt number is same as "Priority Order" in datasheet

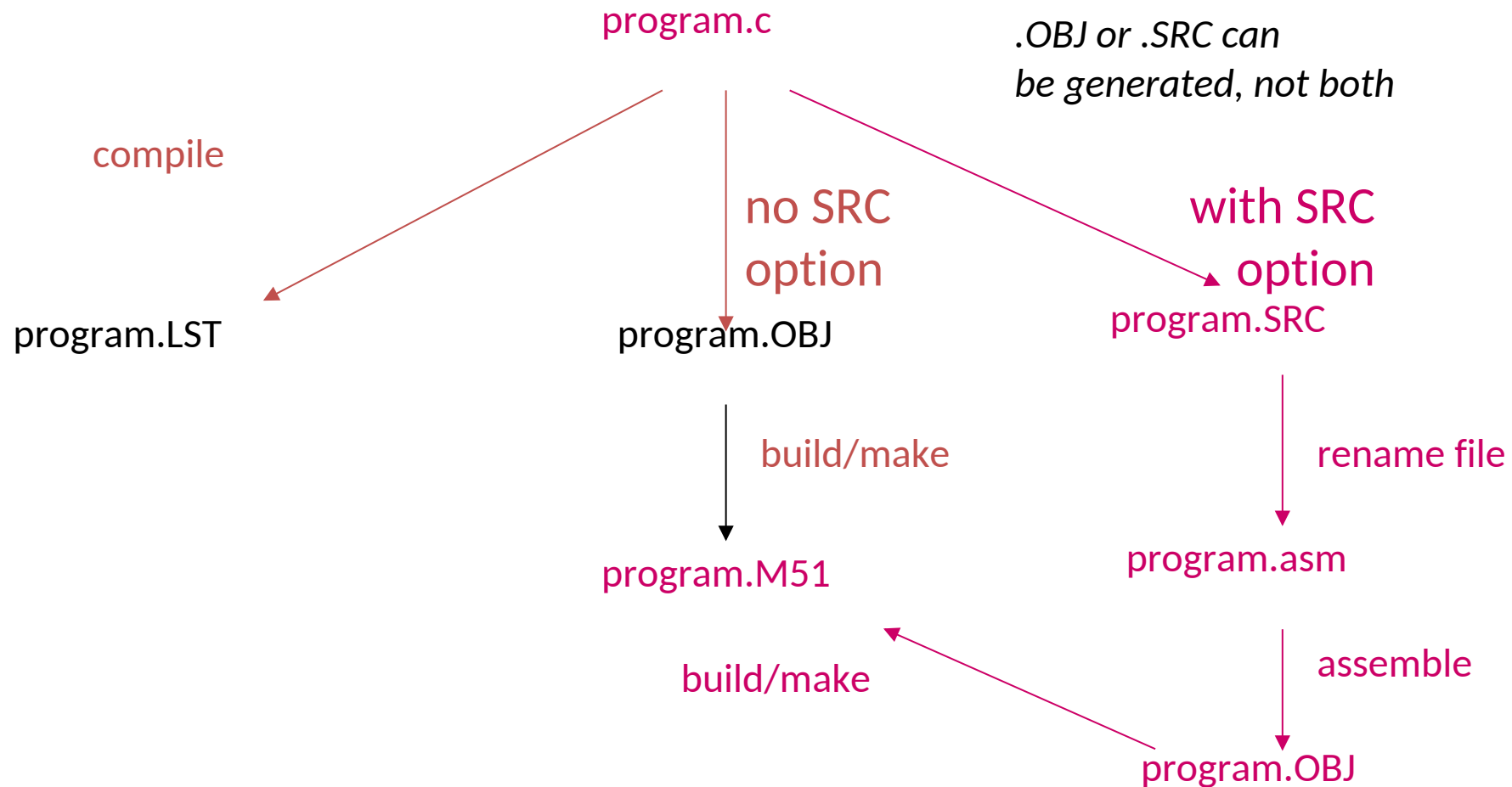
In-line Assembly

Calling Assembly Function from C Code

- When it is more efficient, or easier, can insert assembly code in C programs.

```
#pragma asm  
put your assembly code here  
#pragma endasm
```

Compilation Process (Keil)



Must use this path for C programs with in-line assembly
It is also necessary to add #pragma SRC to code

Example – Switch/LED Program

```
#include <c8051F020.h>
#pragma SRC // Need this to generate .SRC file
void PORT_Init (void);
```

```
char Get_SW(void) {
#pragma ASM
mov a, P3
anl a, #80h // mask all but P3.7
mov R7, a // function value (char) returned in R7
#pragma ENDASM
}
```

```
void Set_LED(void) {
#pragma ASM
setb P1.6
#pragma ENDASM
}
```

Functions can be implemented in assembly language



```
void Clr_LED(void) {
#pragma ASM
clr P1.6
#pragma ENDASM
}
```

```
void PORT_Init (void){ XBR2 = 0x40; // Enable crossbar and enable P1.6 (LED) as push-pull output}
P1MDOUT |= 0x40; // enable P1.6 (LED) as push-pull output
}
```

Main function



```
void main(void) {
PORT_Init();
while (1)
if (Get_SW()) Set_LED();
else Clr_LED();
}
```

Calling Assembly Function from C Code

Make sure you are in debug mode; uncomment the start-up file; must be in LINK mode; connect your board to the PC

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows files like `Combine_Assem_C_Code_2.c`, `system_stm32f4xx.c`, `C_Blink_LED.c`, `main.c`, and `simple_C.c`.
- Code Editor:** Displays the assembly function `sum4` and its call in `main`.

```
3 // Purpose: Combine Assembly and C code together
4 // INPUT: NONE
5 // OUTPUT: None
6 // Dependencies: None
7 // Revision: 1
8 //*****
9 //<include <stdio.h>
10 #include "stm32f4xx.h"
11
12 _asm uint32_t sum4(int a, int b, int c, int d){
13
14     push {r4,lr}
15     // arguments stored in r0-r3
16     MOV r4, r0      ; r0 = 1st argument
17     ADD r4, r4, r1   ; r1 = 2nd argument
18     ADD r4, r4, r2   ; r2 = 3rd argument
19     ADD r0, r4, r3   ; r3 = 4th argument r0 = return
20     pop {r4, pc}
21 }
22
23 int main(void){
24     static int32_t s = 0;
25     s= sum4(2, 2, 3, 6);
26     while(1);
27 }
```
- Watch Window:** Shows variables `s1` (ulong), `s2` (ulong), `res` (uchar), `s` (int), and `s` (int). The `s` (int) entries are circled in red.
- Memory Window:** Shows memory addresses from `0x20000000` to `0x20000024` with their corresponding values.
- Symbols Window:** Shows the symbol table with entries for `main`, `sum4`, and other functions.

Where are the values of a,b,c,d stored?

What happens if you don't perform PUSH or POP?

What is the memory location of variable s?

What happens if you don't uncomment <stdio.h>

Change the value of a = 43. Is the answer correct?

Calling Assembly Function from C Code

Make sure you are in debug mode; uncomment the start-up file; must be in LINK mode; connect your board to the PC

Note that in this case we use inline assembly using virtual registers (a,b,c, d)

The screenshot displays an IDE with several windows. The main window shows a C file named `Combine_Assem_C_Code_2.c` with the following code:

```
6 // Dependencies: None
7 // Revision: 1
8 //*****
9 //include <stdio.h>
10 //include "stm32f4xx.h"
11
12 int sum5(int a, int b, int c, int d){
13     static int32_t t = 0;
14     asm{
15         // arguments stored in r0-r3
16         ADD t, a, b;
17         ADD t, c;
18         ADD t, d;
19     }
20     return t;
21 }
22
23 int main(void){
24     static int32_t s = 0;
25     s = sum5(2, 2, 3, 6);
26     while(1);
27 }
```

The assembly window shows the inline assembly code being executed. The `Watch 1` window shows the following variables:

Name	Value	Type
s1	0x00000000	ulong
s2	0x00000000	ulong
res	<cannot evaluate>	uchar
s	0x0000000D	int
s	0x0000000D	int

The `Memory 1` window shows the memory address `0x20000000` and the following data:

Address	Value
0x20000000	013 000 000 000 013 000
0x20000006	000 000 000 000 000 000
0x2000000C	000 000 000 000 000 000

The `Symbols` window shows the following symbols:

Module / Name	Location	Type
Virtual Registers		
Special Function Registers		
workingProject		Application
<Types>		
RTE/Device/STM32F401RETx/sy...		Module
RTE/Device/STM32F401RETx/st...		Module
Combine_Assem_C_Code_2.c		Module
main	0x08000354	int f()
s	0x20000004	int32_t
sum5	0x0800036C	int f(int,int,l...
a		param int
b		param int
c		param int
d		param int
t	0x20000000	int32_t
__main	0x08000194	function
__scatterload_rt2	0x0800019C	function

Where are the values of a,b,c,d stored?

Are we still using PUSH or POP?

What is the memory location of variable s and t?

What happens if we have `int t = 0` instead?

What happens if we have `int32_t t = 0` instead?

Change the value of a = 43. Is the answer correct?

Simple C Project

Blinking LED PA5

Using Header File

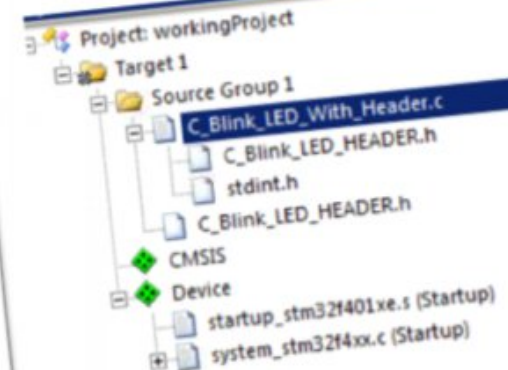
```
1 //*****
2 // C-LED-Blinker
3 // Purpose: C Code Blink LED
4 // INPUT: NONE
5 // OUTPUT: ON-BOARD LED (PORTA-PA5)
6 // Dependencies: Header File
7 // Revision: 1
8 //*****
9
10 #include "stdint.h"
11 #include "C_Blink_LED_HEADER.h"
12
13 // select the I/O direction
14 #define LED_DELAY          500000
15 #define LED_ON              0x0020
16 #define LED_OFF             0x0
17
18 int main(){
19
20     RCC_AHB1ENR_REG |= 0x5;
21     GPIOA_MODER_REG |= 0x400;
22     uint32_t MyTIMER = LED_DELAY;
23
24     while(1){
25
26         GPIOA_ODR_REG ^= LED_ON;
27         while(MyTIMER > 0){ MyTIMER--; }
28         MyTIMER = LED_DELAY;
29     }
30
31     return 0;
32 }
```

```
1 //*****
2 //Header File .h
3 //Header file for the project
4 //*****
5
6 // Defining the Constants
7
8 //-----Related to Clock -----
9 // enables/disables clocks for peripherals
10 #define RCC_AHB1ENR_REG  (*((volatile uint32_t *)0x40023830))
11
12 //-----Related to GPIO Port C-----
13 // sets a pin on port C as input or output
14 #define GPIOC_MODER_REG  (*((volatile uint32_t *)0x40020800))
15
16 // stores the data on the input register
17 #define GPIOC_IDR_REG    (*((volatile uint32_t *)0x40020810))
18
19 //-----Related to GPIO Port A-----
20 // stores the data to be output, it is read/write accessible
21 #define GPIOA_ODR_REG    (*((volatile uint32_t *)0x40020014))
22
23 // sets a pin on port A as input or output
24 #define GPIOA_MODER_REG  (*((volatile uint32_t *)0x40020000))
```

```
// delay time for LED on/off
// Turn LED ON
// Turn LED OFF
```

```
// Turn on GPIOA and GPIOC
// 0xA8000400; // Set GPIOA PA05 as an OUT
// Load initial timer
```

```
// Toggle LED
// Just Delay
// Re-load timer
```



Interfacing with C

- Example: Temperature Sensor program
 - Configures the external oscillator
 - Configures the ADC0 for temp. sensor
 - Configures Port1 so LED can be used
 - Configures Timer3 to synch the ADC0
 - Uses ADC0 ISR to take temperature samples and averages 256 of them and posts average to global variable
 - Main program compares average temp. to room temp. and lights LED if temp is warmer.
 - [Temp 2.c](#)

Make it REAL!

Temperature Conversion

Initialization

- When a C program is compiled, some code is created that runs BEFORE the main program.
- This code clears RAM to zero and initializes your variables. Here is a segment of this code:

```
                                LJMP 0003h
0003:    MOV R0, #7FH
                                CLR A
back:    MOV @R0, A
                                DJNZ R0, back
...
```

Arrays in C

- Useful for storing data

type arr_name[dimension]

↓ ↓ ↓
char temp_array[256]

Array elements are stored in adjacent locations in memory.

temp_array[0]	
temp_array[1]	
temp_array[2]	
temp_array[3]	
...	
temp_array[253]	
temp_array[254]	
temp_array[255]	

Pointers in C

- Pointers are variables that hold memory addresses.
- Specified using * prefix.

```
int *pntr;    // defines a pointer, pntr
```

```
pntr = &var; // assigns address of var to pntr
```

Pointers and Arrays

Note: the name of an array is a pointer to the first element:

`*temp_array` is the same as `temp_array[0]`

So the following are the same:

`n = *temp_array;`

`n = temp_array[0];`

and these are also the same:

`n = *(temp_array+5);`

`n = temp_array[5];`

`temp_array[0]`

`temp_array[1]`

`temp_array[2]`

`temp_array[3]`

...

Arrays

- In watch window, address (pointer) of first element array is shown.
- Array is not initialized as you specify when you download or reset, but it will be when Main starts.

```
unsigned char P0_out[4] = {0x01,0x02,0x04,0x08};
```



P0_out	0x0b
P0_out[0]	18
P0_out[1]	06
P0_out[2]	08
P0_out[3]	76
P0_out[4]	36

Array Example

Compiler Optimization Levels

- Optimization level can be set by compiler control directive:
- Examples (default is #pragma (8, speed))
 - #pragma ot (7)
 - #pragma ot (9, size)
 - #pragma ot (size) – reduce memory used at the expense of speed.
 - #pragma ot (speed) – reduce execution time at the expense of memory.

Compiler Optimization Levels

Level	Optimizations added for that level
0	Constant Folding: The compiler performs calculations that reduce expressions to numeric constants, where possible. This includes calculations of run-time addresses. Simple Access Optimizing: The compiler optimizes access of internal data and bit addresses in the 8051 system. Jump Optimizing: The compiler always extends jumps to the final target. Jumps to jumps are deleted.
1	Dead Code Elimination: Unused code fragments and artifacts are eliminated. Jump Negation: Conditional jumps are closely examined to see if they can be streamlined or eliminated by the inversion of the test logic.
2	
3	
4	
5	
6	
7	
8	
9	Common Block Subroutines: Detects recurring instruction sequences and converts them into subroutines. Cx51 even rearranges code to obtain larger recurring sequences.

Example: 7-seg Decoder

```
// Program to convert 0-F into 7-segment equivalents.
#pragma debug code)
#pragma ot (9)
#include <c8051f020.h>
#define NUM_SAMPLES 16
unsigned char SEGS7[16] = {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92,
0x82, 0xF8, 0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E};

xdata unsigned char samples[NUM_SAMPLES];

void main (void)
{
    char i;                                // loop counter
    WDTCN = 0xde;
    WDTCN = 0xad;
    for (i=0; i < NUM_SAMPLES; i++)
        {samples[i] = SEGS7[i];}
    while (1);
}
```

Effect of Optimization Level on Code Size

Level	Code Size
0	53
1	53
2	53
3	51
4	46
5	46
6	39
7	39
8	38
9	38

Level 0 Optimization

```
; FUNCTION main (BEGIN)
0000 75FFDE          MOV      WDTCN,#0DEH
0003 75FFAD          MOV      WDTCN,#0ADH
;----- Variable 'i' assigned to Register 'R7' -----
0006 750000          R        MOV      i,#00H
0009 C3              CLR      C
000A E500            R        MOV      A,i
000C 6480              XRL      A,#080H
000E 9490              SUBB     A,#090H
0010 5020              JNC      ?C0004
0012 AF00            R        MOV      R7,i
0014 7400            R        MOV      A,#LOW SEGS7
0016 2F              ADD      A,R7
0017 F8              MOV      R0,A
0018 E6              MOV      A,@R0
...

```

Level 9 Optimization

```
; FUNCTION main (BEGIN)
0000 75FFDE          MOV      WDTCN, #0DEH
0003 75FFAD          MOV      WDTCN, #0ADH
;---- Variable 'i' assigned to Register 'R7' ----
0006 E4             CLR      A
0007 FF             MOV      R7, A
0008 7400           R        MOV      A, #LOW SEGS7
000A 2F             ADD      A, R7
000B F8             MOV      R0, A
000C E6             MOV      A, @R0
...
```

Memory Models

- **Small** - places all function variables and local data segments in the internal data memory (RAM) of the 8051 system. This allows very efficient access to data objects (direct and register modes). The address space of the **SMALL** memory model, however, is limited.
- **COMPACT**: variables are stored in paged memory addressed by ports 0 and 2. Indirect addressing opcodes are used. On-chip registers are still used for locals and parameters.
- **Large** - all variables and local data segments of functions and procedures reside (as defined) in the external data memory of the 8051 system. Up to 64 KBytes of external data memory may be accessed. This, however, requires the long and therefore inefficient form of data access through the data pointer (**DPTR**).

- **Selected by compiler directives**

- **Examples:**

- `#pragma small`
 - `#pragma large`

- **Multi-Model Programs**

```
1.#pragma COMPACT
2.void fast_func(void) SMALL{
3./*code*/
```

Single chip 8051 users may only use the SMALL model, unless they have an external RAM fitted which can be page addressed from Port 0 and optionally, Port 2, using MOVX A,@R0 addressing.

This permits the COMPACT model. While it is possible to change the global memory model half way through a project, it is not recommended!

Example: LARGE

0006	E4		CLR	A
0007	FF		MOV	R7, A
0008	EF		MOV	A, R7
0009	FD		MOV	R5, A
000A	33		RLC	A ;multiply by 2
000B	95E0		SUBB	A, ACC
000D	FC		MOV	R4, A
000E	7400	R	MOV	A, #LOW SEGS7
0010	2D		ADD	A, R5
0011	F582		MOV	DPL, A
0013	7400	R	MOV	A, #HIGH SEGS7
0015	3C		ADDC	A, R4
0016	F583		MOV	DPH, A
0018	E0		MOVX	A, @DPTR

... •

Registers R4, R5 keep track of 16-bit data address (external RAM)

Example: SMALL

0006	E4		CLR	A
0007	FF		MOV	R7, A
0008	7400	R	MOV	A, #LOW SEGS7
000A	2F		ADD	A, R7
000B	F8		MOV	R0, A
000C	E6		MOV	A, @R0
... •				

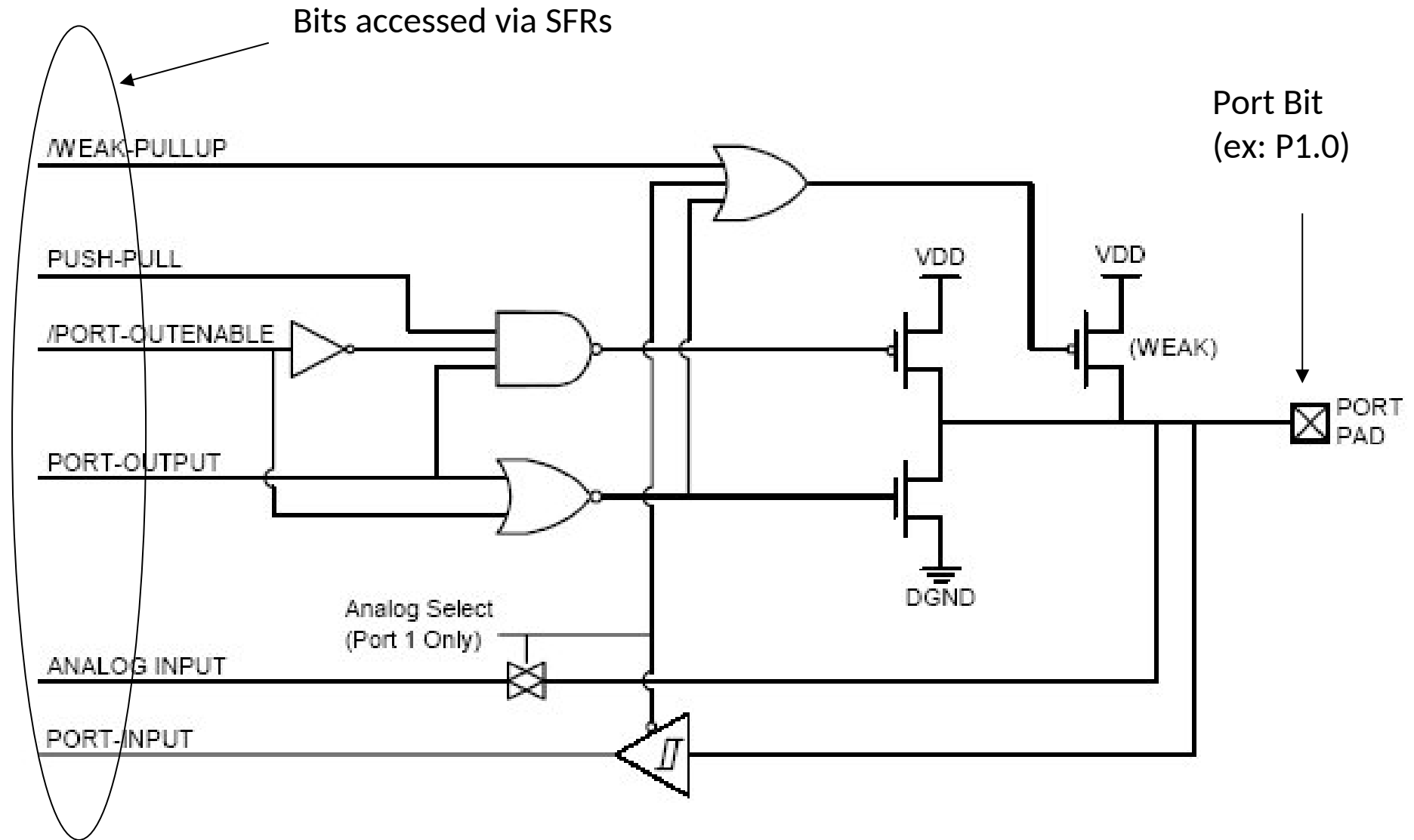
Data address = #LOW SEGS7 + R7 (8-bit address, RAM)

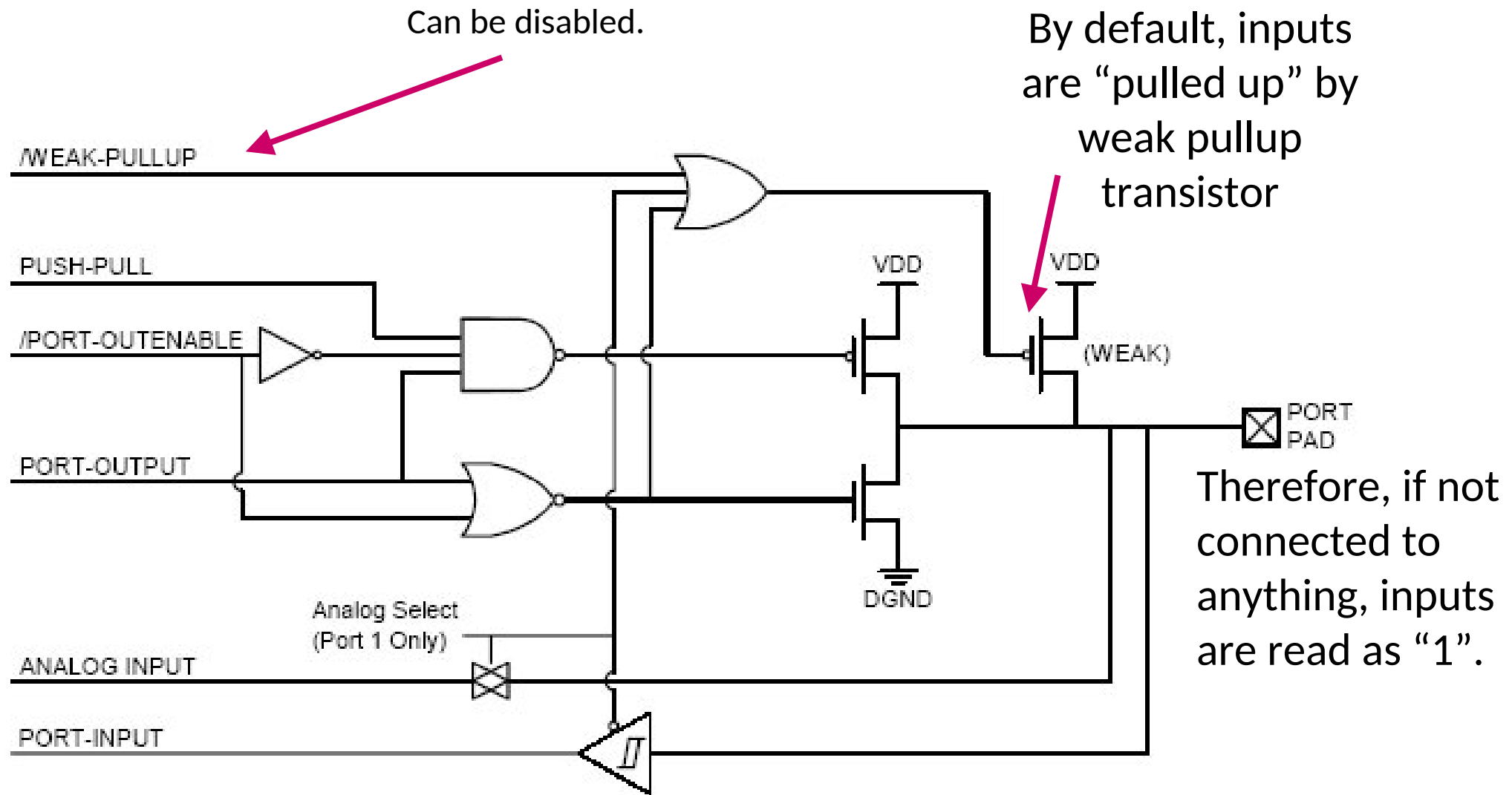
Initialization

- When a C program is compiled, some code is created that runs BEFORE the main program.
- This code clears RAM to zero and initializes your variables. Here is a segment of this code:

```
                                LJMP 0003h
0003:    MOV R0, #7FH
                                CLR A
back:    MOV @R0, A
                                DJNZ R0, back
...
```

I/O Circuitry - Exercise

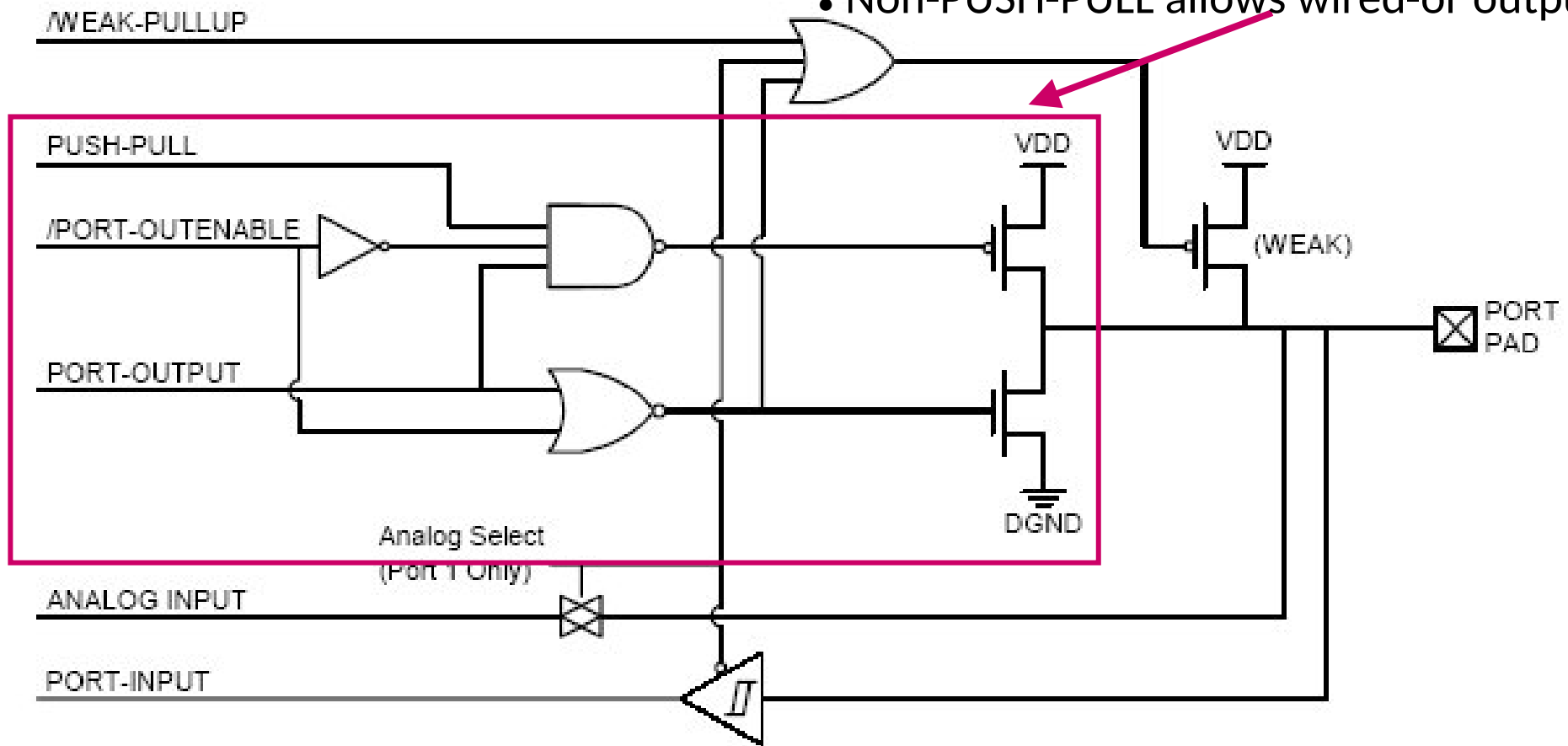




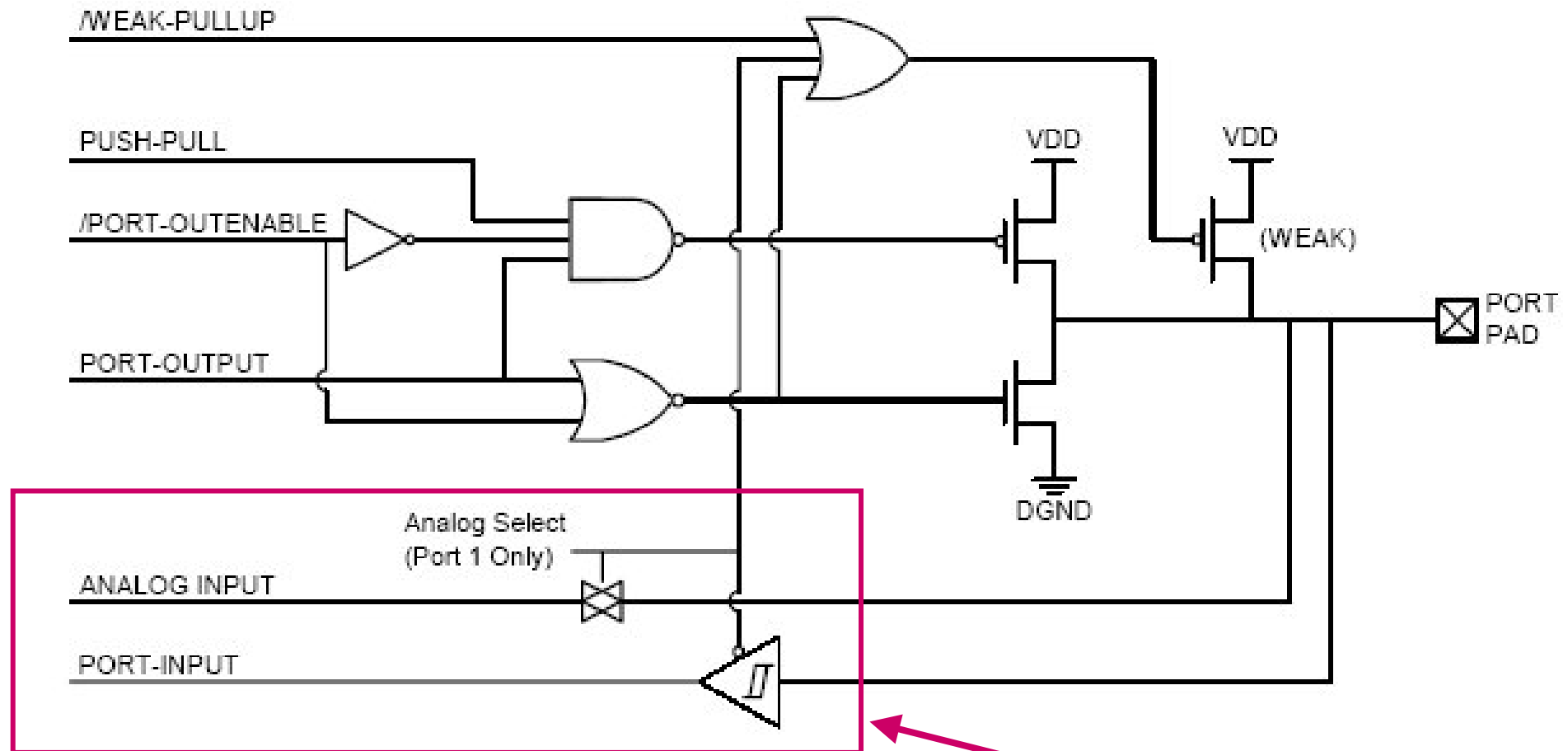
Port I/O - Output

Output circuit:

- Only enabled if /PORT-OUTENABLE = 0
- PUSH-PULL = 1 enables P transistor
- Non-PUSH-PULL allows wired-or outputs



Port I/O - Input



Port 1 can be configured for either digital or analog inputs using a pass transistor and buffer

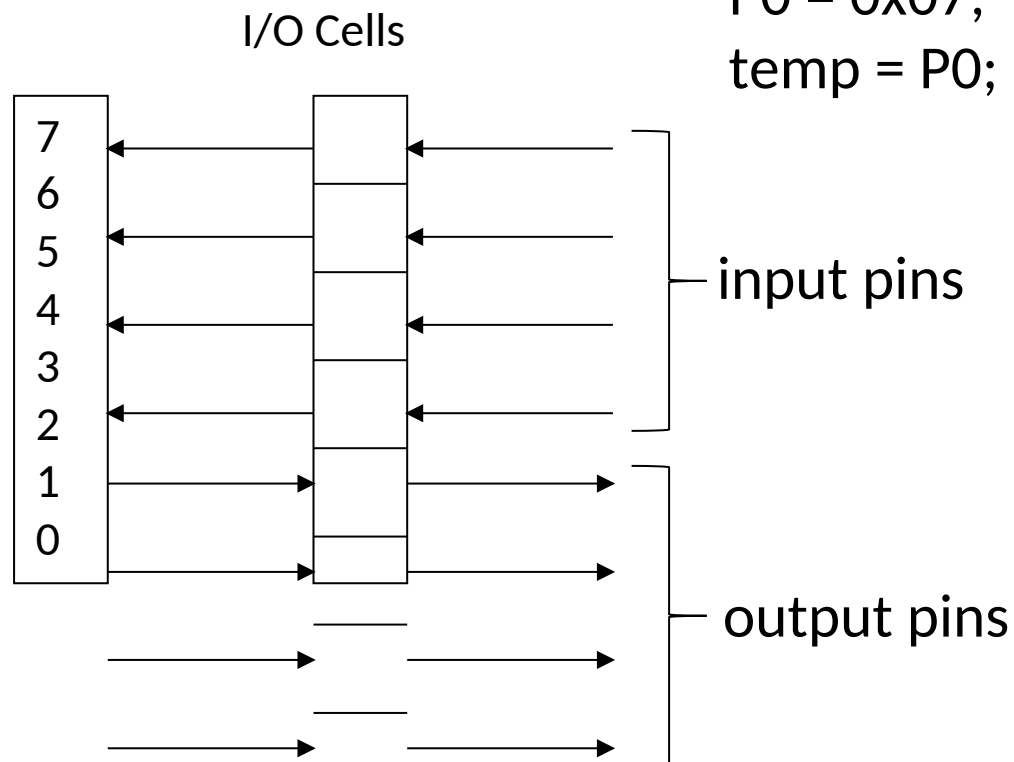
Port I/O Example

```
XBR2 = 0x40;      // Enable XBAR2  
P0MDOUT = 0x0F;   // Outputs on P0 (0-3)
```

...

```
P0 = 0x07;        // Set pins 2,1,0 and clear pin 3  
temp = P0;        // Read Port0
```

Port 0 Latch



Keypad Interface

C for Large Projects

- Use functions to make programs modular
- Break project into separate files if the programs get too large
- Use header (`#include`) files to hold definitions used by several programs
- Keep main program short and easy to follow
- Consider multi-tasking or multi-threaded implementations

Functions

- The basis for modular structured programming in C.

```
return-type  function-name(argument declarations)
{
    declarations and statements
}
```

Example – no return value or arguments

```
void SYSCLK_Init (void) {  
    // Delay counter  
    int i;  
        // Start external oscillator with 22.1184MHz crystal  
        OSCXCN = 0x67;  
        // Wait for XTLVLD blanking interval (>1ms)  
        for (i = 0; i < 256; i++) ;  
        // Wait for crystal osc. to settle  
        while (!(OSCXCN & 0x80)) ;  
        // Select external oscillator as SYSCLK  
        OSCICN = 0x88;  
}
```

Example – with arguments

```
void Timer3_Init (int counts) {  
    // Stop timer, clear TF3, use SYSCLK as timebase  
    TMR3CN = 0x02;  
    // Init reload value  
    TMR3RL = -counts;   
    // Set to reload immediately  
    TMR3 = 0xffff;  
    // Disable interrupts  
    EIE2 &= ~0x01;  
    // Start timer  
    TMR3CN |= 0x04;  
}
```

Example – with return value

```
char ascii_conv (char num) {  
    return num + 30;  
}
```

Header Files

- Use to define global constants and variables

```
// 16-bit SFR Definitions for 'F02x
sfr16 TMR3RL = 0x92;           // Timer3 reload value
sfr16 TMR3 = 0x94;             // Timer3 counter
sfr16 ADC0 = 0xbe;             // ADC0 data
sfr16 DAC0 = 0xd2;             // DAC data
sfr16 DAC1 = 0xd5;
// Global CONSTANTS
#define SYSCLK 22118400         // SYSCLK frequency in Hz
sbit LED = P1^6;               // LED='1' means ON
sbit SW1 = P3^7;               // SW1='0' means switch pressed
#define MAX_DAC ((1<<12)-1)    // Maximum value of the DAC register 12 bits
#define MAX_INTEGRAL (1L<<24)  // Maximum value of the integral
// Function PROTOTYPES
void SYSCLK_Init (void);
void PORT_Init (void);
void ADC0_Init (void);
void DAC_Init (void);
void Timer3_Init (int counts);
void ADC0_ISR (void);
```

Multitasking and Multithreading

- Multitasking: Perception of multiple tasks being executed simultaneously.
 - Usually a feature of an operating system and tasks are separate applications.
 - Embedded systems are usually dedicated to one application.
- Multithreading: Perception of multiple tasks within a single application being executed.
 - Example: Cygnal IDE color codes while echoing characters you type.

Multitasking and Multithreading

A “thread”

```
void main (void) {  
    long temperature;  
    WDTCN = 0xde;  
    WDTCN = 0xad;  
    SYSCLK_Init();  
    PORT_Init();  
    Timer3_Init (SYSCLK/SAMPLE_RATE);  
    ADOEN = 1;  
    EA = 1;  
    while (1) {  
        temperature = result;  
        if (temperature < 0xB230) LED = 0;  
        else LED = 1;  
    }  
}  
  
void SYSCLK_Init (void){  
    int i;  
    OSCXCN = 0x67;  
    for (i=0; i < 256; i++) ;  
    while (!(OSCXCN & 0x80)) ;  
    OSCICN = 0x88; }  
  
void PORT_Init (void) {  
    XBR0 = 0x04;  
    XBR1 = 0x00;  
    XBR2 = 0x40;  
    POMDOUT |= 0x01;  
    P1MDOUT |= 0x40;}  
  
void Timer3_Init (int counts) {  
    TMR3CN = 0x02;  
    TMR3RL = -counts;  
    TMR3 = 0xffff;  
    EIE2 &= ~0x01;  
    TMR3CN |= 0x04; }
```

More C Examples

TIME DELAY

There are two ways to create a time delay in 8051 C

- Using the 8051 timer (Chap. 9) •

Using a simple for loop

be mindful of three factors that can affect the accuracy of the delay

The 8051 design

- The number of machine cycle
- The number of clock periods per machine cycle

The crystal frequency connected to the X1 - X2 input pins

Compiler choice

- C compiler converts the C statements and functions to Assembly language instructions -
- Different compilers produce different code



TIME DELAY (µs)

Write an 8051 C program to toggle bits of P1 continuously forever with some delay.

Solution:

```
//Toggle P1 forever with some delay in between
//"on" and "off"
#include <reg51.h>
void main(void)
{
    unsigned int x;
    for (;;)                                //repeat forever
    {
        p1=0x55;

        for
            (x=0;x<40000;x++);              //delay size
        p1=0xAA;
        for (x=0;x<40000;x++);
    }
}
```

We must use the oscilloscope to measure the exact duration



TIME DELAY (cont')

Write an 8051 C program to toggle bits of P1 ports continuously with a 250 ms.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
void main(void)
{
    while (1)                                //repeat forever
    {
        p1=0x55;
        MSDelay(250);
        p1=0xAA;
        MSDelay(250);
    }
}

void MSDelay(unsigned int
itime) {
    unsigned int i,j; for
    (i=0;i<itime;i++) for
    (j=0;j<1275;j++); }
```



I/O PROGRAMMING G Byte Size I/O

LEDs are connected to bits P1 and P2. Write an 8051 C program that shows the count from 0 to FFH (0000 0000 to 1111 1111 in binary) on the LEDs.

Solution:

```
#include <reg51.h>
#define LED P2;

void main(void)
{
    P1=00;           //clear P1
    LED=0;           //clear P2
    for (;;)         //repeat forever
    {
        P1++;        //increment P1
        LED++;        //increment P2
    }
}
```

Ports P0 - P3 are byte-accessable and we use the P0 - P3 labels as defined in the 8051/52 header file.



I/O PROGRAMMING

Byte Size I/O# (cont')

Write an 8051 C program to get a byte of data from P1, wait 1/2 second, and then send it to P2.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);

void
main(void) {
    unsigned char mybyte;
    P1=0xFF;           //make P1 input port
    while
    (1) {
        mybyte=P1;     //get a byte from P1
        MSDelay(500);
        P2=mybyte;     //send it to P2
    }
}
```



I/O PROGRAMMING G Byte Size I/O

```
main(void) {
```

Write an 8051 C program to get a byte of data form P0. If it is less than 100, send it to P1; otherwise, send it to P2.

Solution:

```
#include <reg51.h>

oid

    unsigned char mybyte;
    P0=0xFF;           //make P0 input port
    while
    (1) {
        mybyte=P0;     //get a byte from P0
        if (mybyte<100)
            P1=mybyte;  //send it to P1
        else
            P2=mybyte;  //send it to P2
    }
}
```



I/O PROGRAMMING Bit-addressable I/O

Write an 8051 C program to toggle only bit P2.4 continuously without disturbing the rest of the bits of P2.

Solution:

```
//Toggling an individual bit  
#include <reg51.h>  
sbit mybit=P2^4;
```

```
void main(void)  
{  
    while (1)  
    {  
        mybit=1;  
        mybit=0;  
    }  
}
```

Ports P0 - P3 are bit-addressable and we use sbit data type to access a single bit of P0 - P3

Use the Px^y format, where x is the port 0, 1, 2, or 3 and y is the bit 0 - 7 of that port

```
//turn on P2.4  
//turn off P2.4
```



I/O PROGRAMMING

Bit-addressable

I/O
(cont')

Write an 8051 C program to monitor bit P1.5. If it is high, send 55H to P0; otherwise, send AAH to P2.

Solution:

```
#include <reg51.h>
sbit mybit=P1^5;

void
main(void) {
    mybit=1;                                //make mybit an input
    while
    (1) {
        if
        (mybit==1)
            P0=0x55;
        else
            P2=0xAA;
    }
}
```



I/O PROGRAMMING G Bit-addressable I/O (cont')

A door sensor is connected to the P1.1 pin, and a buzzer is connected to P1.7. Write an 8051 C program to monitor the door sensor, and when it opens, sound the buzzer. You can sound the buzzer by sending a square wave of a few hundred Hz.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);
sbit Dsensor=P1^1;
sbit Buzzer=P1^7;

void
main(void) {
    Dsensor=1;                                //make P1.1 an input
    while
    (1) {
        while (Dsensor==1) //while it
        opens {
            Buzzer=0;
            MSDelay(200);
            Buzzer=1;
            MSDelay(200);
        }
    }
}
```



I/O
PROGRAMMING
Bit-addressable
I/O
(cont')

The data pins of an LCD are connected to P1. The information is latched into the LCD whenever its Enable pin goes from high to low. Write an 8051 C program to send "The Earth is but One Country" to this LCD.

Solution:

```
#include <reg51.h>
#define LCDDData P1    //LCDDData declaration
sbit En=P2^0;          //the enable pin

void
main(void) {
    unsigned char message[]
    ="The Earth is but One Country"; unsigned
        char z;
        for      (z=0;z<28;z++)      //send      28
characters {
    LCDDData=message[z];
    En=1;      //a high-
    En=0;      //-to-low pulse to latch data
    }
}
```



I/O PROGRAMMING Accessing SFR Addresses 80 - FFH

Write an 8051 C program to toggle all the bits of P0, P1, and P2 continuously with a 250 ms delay. Use the sfr keyword to declare the port addresses.

Solution:

Another way to access the SFR RAM space 80 - FFH is to use the sfr data type

```
//Accessing Ports as SFRs using sfr data type
sfr P0=0x80;
sfr P1=0x90;
sfr P2=0xA0;
void MSDelay(unsigned int);

void
main(void) {
    while
        (1) {
            P0=0x55;
            P1=0x55;
            P2=0x55;
            MSDelay(250);
            P0=0xAA;
            P1=0xAA;
            P2=0xAA;
            MSDelay(250);
        }
}
```



I/O

Write an 8051 C program to turn bit P1.5 on and off 50,000 times.

Solution:

```
sbit MYBIT=0x95;

void main(void)
{
    unsigned int z;
    for (z=0; z<50000; z++)
    {
        MYBIT=1;
        MYBIT=0; }
}
```

We can access a single bit of any SFR if we specify the bit address

Accessing
SFR Addresses
80 - FFH
(cont')

Notice that there is no `#include<reg51.h>`. This allows us to access any byte of the SFR RAM space 80 - FFH. This is widely used for the new generation of 8051 microcontrollers



I/O PROGRAMMING Using bit Data Type for Bit-addressable RAM

Write an 8051 C program to get the status of bit P1.0, save it, and send it to P2.7 continuously.

Solution:

```
#include <reg51.h>
sbit inbit=P1^0;
sbit outbit=P2^7;
bit membit;                                     //use bit to declare
                                                //bit- addressable memory

void main(void)
{
    while (1)
    {
        membit=inbit;        //get a bit from P1.0
        outbit=membit;       //send it to P2.7
    }
}
```

We use bit data type to access data in a bit-addressable section of the data RAM space 20 - 2FH



LOGIC OPERATIONS

Bit-wise Operators in C

Logical operators

- AND (&&), OR (||), and NOT (!)

Bit-wise operators

- AND (&), OR (|), EX-OR (^), Inverter (~), Shift Right (>>), and Shift Left (<<)

These operators are widely used in software engineering for embedded systems and control

Bit-wise Logic Operators for C

		AND	OR	EX-	Inverter
A	B	A&B	A B	A^B	~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	
1	1	1	1	0	



LOGIC

Run the following program on your simulator and examine the results.

Bit-wise Operators in C (cont')

Solution:

```
#include <reg51.h>

void main(void)
{
    P0=0x35 & 0x0F;           //ANDing
    P1=0x04 | 0x68;           //ORing
    P2=0x54 ^ 0x78;           //XORing
    P0=~0x55;                 //inversing
    P1=0x9A >> 3;             //shifting right 3
    P2=0x77 >> 4;             //shifting right 4
    P0=0x6 << 4;              //shifting left 4
}
```



LOGIC OPERATIONS

Bit-wise Operators in C (cont')

Write an 8051 C program to toggle all the bits of P0 and P2 continuously with a 250 ms delay. Using the inverting and Ex-OR operators, respectively.

Solution:

```
#include <reg51.h>
void MSDelay(unsigned int);

void
main(void) {
    P0=0x55;
    P2=0x55;
    while (1)
    {
        P0=~P0;
        P2=P2^0xFF;
        MSDelay(250); }
}
```



LOGIC OPERATIONS S

Bit-wise Operators in C (cont')

Write an 8051 C program to get bit P1.0 and send it to P2.7 after inverting it.

Solution:

```
#include <reg51.h>
sbit inbit=P1^0;
sbit outbit=P2^7;
bit membit;

void
main(void) {
    while
    (1) {
        membit=inbit;//get a bit from P1.0
        outbit=~membit; //invert it and send
        //it to P2.7
    }
}
```



LOGIC OPERATION S

Bit-wise

send '2' to P0 Operat
send '3' to P0 (o

Write an 8051 C program to read the P1.0 and P1.1 bits and issue an ASCII character to P0 according to the following table.

P1.1	P1.0	
0	0	send '0' to P0
0	1	send '1' to P0
1	0	
1	1	

Solution:

```
#include <reg51.h>

void
main(void) {
    unsigned char z;
    z=P1;
    z=z&0x3;
```



LOGIC OPERATIONS S

Bit-wise Operators in C (cont')

```
switch (z)
{
    case (0) :
    {
        P0 = '0';
        break;
    }
    case (1) :
    {
        P0 = '1';
        break;
    }
    case (2) :
    {
        P0 = '2';
        break;
    }
    case (3) :
    {
        P0 = '3';
        break;
    }
}
```



DATA CONVERSION

Packed BCD to ASCII Conversion

Write an 8051 C program to convert packed BCD 0x29 to ASCII and display the bytes on P1 and P2.

Solution:

```
#include <reg51.h>

void main(void)
{
    unsigned char x,y,z;
    unsigned char mybyte=0x29;
    x=mybyte&0x0F;

    P1=x|0x30;
    y=mybyte&0xF0;
    y=y>>4;
    P2=y|0x30;
}
```



DATA CONVERSION

ASCII to Packed BCD Conversion

Write an 8051 C program to convert ASCII digits of '4' and '7' to packed BCD and display them on P1.

Solution:

```
#include <reg51.h>

void main(void)
{
    unsigned char bcdbyte;
    unsigned char w='4';
    unsigned char z='7';
    w=w&0x0F;
    w=w<<4;
    z=z&0x0F;
    bcdbyte=w|z;
    P1=bcdbyte; }
```



DATA CONVERSION N Checksum Byte in ROM

Write an 8051 C program to calculate the checksum byte for the data 25H, 62H, 3FH, and 52H.

Solution:

```
#include <reg51.h>

void main(void)
{
    unsigned char mydata[]={0x25,0x62,0x3F,0x52};
    unsigned char sum=0;
    unsigned char x;
    unsigned char chksumbyte;
    for (x=0;x<4;x++)
    {
        P2=mydata[x];
        sum=sum+mydata[x];
        P1=sum;
    }
    chksumbyte=~sum+1;
    P1=chksumbyte; }
```



DATA CONVERSION

Checksum Byte in ROM (cont')

Write an 8051 C program to perform the checksum operation to ensure data integrity. If data is good, send ASCII character 'G' to P0. Otherwise send 'B' to P0.

Solution:

```
#include <reg51.h>

void
main(void) {
    unsigned char mydata[]
    ={0x25,0x62,0x3F,0x52,0xE8}; unsigned char
    shksum=0; unsigned char x;

    for (x=0;x<5;x++)
        chksum=chksum+mydata[x];
        if (chksum==0)
            P0='G';
        else
            P0='B';
}
```



DATA CONVERSION

Binary (hex) to
Decimal and
ASCII
Conversion

Write an 8051 C program to convert 11111101 (FD hex) to decimal and display the digits on P0, P1 and P2.

Solution:

```
#include <reg51.h>

void main(void)
{
    unsigned char
    x, binbyte, d1, d2, d3;
    binbyte = 0xFD;
    x = binbyte / 10;
    d1 = binbyte % 10;
    d2 = x % 10;
    d3 = x / 10;
    P0 = d1;
    P1 = d2;
    P2 = d3; }

```



ACCESSIN
CODE
ROM
RAM Data
Space Usage
by 8051 C
Compiler

The 8051 C compiler allocates RAM locations

- Bank 0 - addresses 0 - 7
- Individual variables - addresses 08 and beyond
- Array elements - addresses right after variables

Array elements need contiguous RAM locations and that limits the size of the array due to the fact that we have only 128 bytes of RAM for everything

- Stack - addresses right after array elements



ACCESSING CODE ROM RAM Data Space Usage by 8051 C Compiler (cont')

Compile and single-step the following program on your 8051 simulator. Examine the contents of the 128-byte RAM space to locate the ASCII values.

Solution:

```
#include <reg51.h>

void
main(void) {
    unsigned char mynum[]="ABCDEF"; //RAM space
    unsigned char z;
    for (z=0;z<=6;z++)
        P1=mynum[z];
}
```



ACCESSING
CODE
ROM
RAM Data
Space Usage
by 8051 C
Compiler
(cont')

Write, compile and single-step the following program on your 8051 simulator. Examine the contents of the code space to locate the values.

Solution:

```
#include <reg51.h>

void main(void)
{
    unsigned char mydata[100]; //RAM space
    unsigned char x,z=0;
    for
    (x=0;x<100;x++) {
        z--;
        mydata[x]=z;
        P1=z;
    }
}
```



ACCESSIN
CODE
ROM
8052 RAM Data
Space

One of the new features of the 8052 was an extra 128 bytes of RAM space

- The extra 128 bytes of RAM helps the 8051/52 C compiler to manage its registers and resources much more effectively

We compile the C programs for the 8052 microcontroller

- Use the reg52.h header file
 - Choose the8052 option when compiling the program



ACCESSING CODE ROM (Cont')

Compile and single-step the following program on your 8051 simulator. Examine the contents of the code space to locate the ASCII values.

Solution:

```
#include <reg51.h>

void main(void)
{
    code unsigned char mynum[]="ABCDEF";
    unsigned char z;
    for
    (z=0; z<=6; z++)
    } P1=mynum[z];
```

To make the C compiler use the code space instead of the RAM space, we need to put the keyword `code` in front of the variable declaration



ACCESSING CODE FROM (Cont')

Compare and contrast the following programs and discuss the advantages and disadvantages of each one.

(a)

```
#include <reg51.h>
void main(void)
{
    P1= 'H' ;
    P1= 'E' ;
    P1= 'L' ;
    P1= 'L' ;
    P1= 'O' ; }

```

Short and simple, but the individual characters are embedded into the program and it mixes the code and data together



ACCESSING CODE ROM (cont')

(b)

```
#include <reg51.h>
void
main(void) {
    unsigned char mydata[]="HELLO";
    unsigned char z;
    for (z=0; z<=5; z++)
        P1=mydata[z];
}
```

Use the RAM data space to store array elements, therefore the size of the array is limited

(c)

```
#include <reg51.h>
void
main(void) {
    code unsigned char mydata[]="HELLO";
    unsigned char z;
    for
        (z=0; z<=5; z++)
    } P1=mydata[z];
```

Use a separate area of the code space for data. This allows the size of the array to be as long as you want if you have the on-chip ROM.

However, the more code space you use for data, the less space is left for your program code



DATA SERIALIZATION

Serializing data is a way of sending a byte of data one bit at a time through a single pin of microcontroller

- Using the serial port (Chap. 10) •

Transfer data one bit a time and control the sequence of data and spaces in between them

In many new generations of devices such as LCD, ADC, and ROM the serial versions are becoming popular since they take less space on a PCB



DATA SERIALIZATION (cont')

Write a C program to send out the value 44H serially one bit at a time via P1.0. The LSB should go out first.

Solution:

```
#include <reg51.h>
sbit P1b0=P1^0;
sbit regALSB=ACC^0;

void
main(void) {
    unsigned char conbyte=0x44;
    unsigned char x;
    ACC=conbyte;
    for
    (x=0;x<8;x++) {
        P1b0=regALSB;
        ACC=ACC>>1; }

}
```



DATA SERIALIZATION (cont')

Write a C program to send out the value 44H serially one bit at a time via P1.0. The MSB should go out first.

Solution:

```
#include <reg51.h>
sbit P1b0=P1^0;
sbit regAMSB=ACC^7;

void
main(void) {
    unsigned char conbyte=0x44;
    unsigned char x;
    ACC=conbyte;
    for
    (x=0;x<8;x++) {
        P1b0=regAMSB;
        ACC=ACC<<1; }

}
```



DATA SERIALIZATION (cont')

Write a C program to bring in a byte of data serially one bit at a time via P1.0. The LSB should come in first.

Solution:

```
#include <reg51.h>
sbit P1b0=P1^0;
sbit ACCMSB=ACC^7;
bit membit;

void
main(void) {
    unsigned char x;
    for (x=0;x<8;x++)
    {
        membit=P1b0;
        ACC=ACC>>1;
        ACCMSB=membit; }

    P2=ACC;
}
```



DATA SERIALIZATION (cont')

Write a C program to bring in a byte of data serially one bit at a time via P1.0. The MSB should come in first.

Solution:

```
#include <reg51.h>
sbit P1b0=P1^0;
sbit regALSB=ACC^0;
bit membit;

void
main(void) {
    unsigned char x;
    for (x=0;x<8;x++)
    {
        membit=P1b0;
        ACC=ACC<<1;
        regALSB=membit;
    }
    P2=ACC;
}
```



PROGRAMMING TIMER/COUNTER

Write an 8051 C

TIMERS IN C

Accessing Timer Registers

Example 9-20

program to toggle all the bits of port P1 continuously with some delay in between. Use Timer 0, 16-bit mode to generate the delay.

Solution:

```
#include <reg51.h>
void T0Delay(void);
void main(void) {
    while (1) {
        P1=0x55;
        T0Delay();
        P1=0xAA;
        T0Delay(); }
}

void
T0Delay() { TMOD
=0x01; TH0=0x00;
TH0=0x35;
TR0=1;
while (TF0==0);
TR0=0;
TF0=0;
}
```

$FFFFH - 3500H = CAFFH$

$= 51967 + 1 = 51968$

$51968 \times 1.085 \mu s = 56.384 \text{ ms}$ is the approximate delay



PROGRAMMING TIMERS IN C

Calculating Delay Length Using Timers



To speed up the 8051, many recent versions of the 8051 have reduced the number of clocks per machine cycle from 12 to four, or even one. The frequency for the timer is always $1/12^{\text{th}}$ the frequency of the crystal attached to the 8051, regardless of the 8051 version.

Write an 8051 C

TIMERS IN C

Times 0/1
Delay Using
Mode 1 (16-bit
Non Auto-
reload)



Example 9-21

program to toggle only bit P1.5 continuously every 0 ms. Use Timer 0, mode 1 (16-bit) to create the delay. Test the program on the (a) AT89C51 and (b) DS89C420.

Solution:

```
#include <reg51.h>
void T0M1Delay(void);
sbit mybit=P1^5;
void main(void){
    while (1)
    { mybit=~mybit;
      T0M1Delay();
    }
}

void
T0M1Delay(void){ TMOD
=0x01; TH0=0xFD;
TH0=0x4B;
TR0=1;
while (TF0==0);
TR0=0;
TF0=0; }
```

$FFFFH - 4BFDH = B402H$
 $= 46082 + 1 = 46083$
 $46083 \times 1.085 \mu s = 50 ms$

PROGRAMMING TIMERS IN C

Times 0/1
Delay Using
Mode 1 (16-bit
Non Auto-
reload)
(cont')

Example 9-22

Write an 8051 C program to toggle all bits of P2 continuously every 500 ms. Use Timer 1, mode 1 to create the delay.

Solution:

//tested for DS89C420, XTAL = 11.0592 MHz

```
#include <reg51.h>
void T1M1Delay(void);
void main(void) {
    unsigned char x;
    P2=0x55;
    while (1) {
        P2=~P2;
        for (x=0;x<20;x++)
            T1M1Delay();
    }
}

void
T1M1Delay(void) {    TMOD
=0x10;    TH1=0xFE;
    TH1=0xA5;
    TR1=1;
    while (TF1==0);
    TR1=0;
    TF1=0;
}
```

A5FEH = 42494 in decimal
 $65536 - 42494 = 23042$
 $23042 \times 1.085 \mu\text{s} = 25 \text{ ms}$ and
 $20 \times 25 \text{ ms} = 500 \text{ ms}$



PROGRAMMING TIMERS IN C

Times 0/1
Delay Using
Mode 1 (16-bit
Non Auto-
reload)
(cont')



Example 9-25

A switch is connected to pin P1.2. Write an 8051 C program to monitor SW and create the following frequencies on pin P1.7:

SW=0: 500Hz

SW=1: 750Hz, use Timer 0, mode 1 for both of them.

Solution:

```
#include <reg51.h>
sbit mybit=P1^5;
sbit SW=P1^7;
void T0M1Delay(unsigned char);
void main(void) {
    SW=1;
    while (1) {
        mybit=~mybit; if
        (SW==0)
            T0M1Delay(0);
        else
            T0M1Delay(1);
    }
}
```

PROGRAMMING TIMERS IN C

Times 0/1
Delay Using
Mode 1 (16-bit
Non Auto-
reload)
(cont')

Example 9-25

```
void T0M1Delay(unsigned char  
c) { TMOD=0x01;  
  if (c==0) {  
    TL0=0x67;  
    TH0=0xFC;  
  }  
  else {  
    TL0=0x9A;  
    TH0=0xFD;  
  }  
  TR0=1;  
  while (TF0==0);  
  TR0=0;  
  TF0=0;  
}
```

FC67H = 64615

65536 - 64615 = 921

$921 \times 1.085 \mu\text{s} = 999.285 \mu\text{s}$

$1 / (999.285 \mu\text{s} \times 2) = 500 \text{ Hz}$



PROGRAMMING TIMERS IN C

Times 0/1 Delay Using Mode 2 (8-bit Auto-reload)

Example 9-23

Write an 8051 C program to toggle only pin P1.5 continuously every 250 ms. Use Timer 0, mode 2 (8-bit auto-reload) to create the delay.

Solution:

```
#include <reg51.h>
void T0M2Delay(void);
sbit mybit=P1^5;
void main(void){
    unsigned char x,y;
    while (1) {
        mybit=~mybit;
        for (x=0;x<250;x++)
            for (y=0;y<36;y++) //we put 36,
                                not 40 T0M2Delay();
    }
}

void
T0M2Delay(void){ TMOD
    TH0=-23;
    TR0=1;
    while (TF0==0);
    TR0=0;
    TF0=0;
}
```

Due to overhead of the for loop
in C, we put 36 instead of 40

$256 - 23 = 233$
 $23 \times 1.085 \mu s = 25 \mu s$ and
 $25 \mu s \times 250 \times 40 = 250 \text{ ms}$



PROGRAMMING TIMERS IN C

Times 0/1 Delay Using Mode 2 (8-bit Auto-reload) (cont')

Example 9-24

Write an 8051 C program to create a frequency of 2500 Hz on pin P2.7. Use Timer 1, mode 2 to create delay.

Solution:

```
#include <reg51.h>
void T1M2Delay(void);
sbit mybit=P2^7;
void main(void){
    unsigned char x;
    while (1) {
        mybit=~mybit;
        T1M2Delay();
    }
}

void
T1M2Delay(void){ TMOD
=0x20; TH1=-184;
TR1=1;
while (TF1==0);
TR1=0;
TF1=0;
}
```

$$\begin{aligned} 1/2500 \text{ Hz} &= 400 \mu\text{s} \\ 400 \mu\text{s} / 2 &= 200 \mu\text{s} \\ 200 \mu\text{s} / 1.085 \mu\text{s} &= 184 \end{aligned}$$



PROGRAMMING TIMERS IN C

C Programming of Timers as Counters

Example 9-26

Assume that a 1-Hz external clock is being fed into pin T1 (P3.5). Write a C program for counter 1 in mode 2 (8-bit auto reload) to count up and display the state of the TL1 count on P1. Start the count at 0H.

Solution:

```
#include <reg51.h>
sbit      T1=P3^5;
void main(void) {
    T1=1;
    TMOD=0x60;
    TH1=0;
    while (1)
    { do {
        TR1=1;
        P1=TL1;
    }
    while (TF1==0);
    TR1=0;
    TF1=0;
    }
}
```



PROGRAMMING TIMERS IN C

C Programming of Timers as Counters (cont')



Example 9-27

Assume that a 1-Hz external clock is being fed into pin T0 (P3.4). Write a C program for counter 0 in mode 1 (16-bit) to count the pulses and display the state of the TH0 and TL0 registers on P2 and P1, respectively.

Solution:

```
#include <reg51.h>
void main(void) {
    T0=1;
    TMOD=0x05;
    TL0=0;
    TH0=0;
    while (1)
    { do {
        TR0=1;
        P1=TL0;
        P2=TH0; }
      while (TF0==0);
      TR0=0;
      TF0=0;
    }
}
```

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references